

High performance E-Commerce

إعداد :

- نغم خلدون مجاهد
- رغد مازن الخوص
- زياد راتب سليمان
- زين علاء الدين رحمون
- محمد نور شيخموس شيخموس

رابط الغيت :

<https://github.com/NaghamMujahed/High-performance-E-Commerce>

بإشراف المهندس حذيفة

الطلب الاول :

1.المشكلة

عند وصول عدة مستخدمين بنفس الوقت لتعديل كمية منتج في المخزون، يحدث تضارب في البيانات (Race condition)

مثال:

- كمية منتج معين 1
- مستخدم A قرأ الكمية يلي بتساوي 1
- مستخدم B قرأ الكمية يلي بتساوي 1
- مستخدم A اشترى <== الكمية صارت 0
- مستخدم B اشترى كمان <== الكمية صارت -1

النتيجة :بيانات خاطئة وخسارة في المخزون.

2.الحلول المقترحة

الحل الأول (بدون حماية)

- كل مستخدم يقرأ ويعدل بحرية
- لا يوجد أي تحكم في الوصول المتزامن
- النتيجة :تضارب وبيانات خاطئة

الحل الثاني Optimistic Locking

- يسمح لجميع المستخدمين بالقراءة بحرية
- يضيف حقن Version لكل سجل
- عند الحفظ يتحقق إذا تغيرت البيانات
- إذا تغيرت يرفع خطأOptimisticLockException
- مناسب للأنظمة التي تكون فيها القراءة أكثر من الكتابة

الحل الثالث ال synchronized

- يضمن أن thread واحد فقط يستطيع تنفيذ الكود المحمي في نفس الوقت بإضافة كلمة synchronize
- سبب عدم اختياره لأنه يعمل على مستوى الكود فقط
- يعني إذا كان النظام يعمل على أكثر من سيرفر فكل سيرفر سيكون له قفله الخاص ولا يعلم بالسيرفرات الأخرى مما يجعله غير كاف لحماية البيانات

الحل الرابع (المختار) Pessimistic Locking

- يقفل السجل عند القراءة
- لا يسمح لأي مستخدم آخر بالقراءة أو التعديل حتى ينتهي الأول
- يضمن عدم حدوث أي تضارب في البيانات
- مناسب للبيانات الحساسة مثل المخزون
- يحمي البيانات من التضارب على مستوى قاعدة البيانات DB

3. سبب اختيار Pessimistic Locking

تم اختيار Pessimistic Locking لعدة أسباب:

- المخزون بيانات حساسة جداً — لا يمكن قبول أي خطأ
- لا يمكن السماح بقراءة بيانات قديمة (Dirty Read)
- Spring Data JPA يدعمه مباشرة بـ @Lock
- يضمن سلامة البيانات تحت الضغط العالي

4. نتائج الاختبار بـ Jmeter و ال AOP

تم اختبار النظام بـ 100 مستخدم متزامن على نفس المنتج:

قبل التحسين

صورة 1 :

من التيرمنال باستخدام مفهوم ال (LOG) AOP

نوضح في هذا الصورة كل ثريد متى انتهى وما هو الوقت الذي استغرقة وانتهى فيه

```
Terminal: Local + -
2026-05-15T13:54:14.293+03:00 INFO 15724 --- [demo] [o-8080-exec-100] c.example.demo.aspect.PerformanceAspect : [PURCHASE] Started: purchaseWithoutLock | Thread: http-nio-8080-exec-100
2026-05-15T13:54:14.318+03:00 INFO 15724 --- [demo] [io-8080-exec-24] c.example.demo.aspect.PerformanceAspect : [PURCHASE] Finished: purchaseWithoutLock | Time: 1190ms | Thread: http-nio-8080-exec-24
2026-05-15T13:54:14.334+03:00 INFO 15724 --- [demo] [nio-8080-exec-9] c.example.demo.aspect.PerformanceAspect : [PURCHASE] Finished: purchaseWithoutLock | Time: 1224ms | Thread: http-nio-8080-exec-9
2026-05-15T13:54:14.494+03:00 INFO 15724 --- [demo] [nio-8080-exec-9] c.example.demo.aspect.PerformanceAspect : Finished: ProductService.purchaseWithoutLock | Time: 1421ms
2026-05-15T13:54:14.348+03:00 INFO 15724 --- [demo] [nio-8080-exec-4] c.example.demo.aspect.PerformanceAspect : Finished: ProductService.purchaseWithoutLock | Time: 1263ms
2026-05-15T13:54:14.375+03:00 INFO 15724 --- [demo] [nio-8080-exec-2] c.example.demo.aspect.PerformanceAspect : [PURCHASE] Finished: purchaseWithoutLock | Time: 1261ms | Thread: http-nio-8080-exec-2
2026-05-15T13:54:14.497+03:00 INFO 15724 --- [demo] [nio-8080-exec-2] c.example.demo.aspect.PerformanceAspect : Finished: ProductService.purchaseWithoutLock | Time: 1421ms
2026-05-15T13:54:14.458+03:00 INFO 15724 --- [demo] [io-8080-exec-21] c.example.demo.aspect.PerformanceAspect : [PURCHASE] Finished: purchaseWithoutLock | Time: 1347ms | Thread: http-nio-8080-exec-21
2026-05-15T13:54:14.501+03:00 INFO 15724 --- [demo] [io-8080-exec-21] c.example.demo.aspect.PerformanceAspect : Finished: ProductService.purchaseWithoutLock | Time: 1426ms
2026-05-15T13:54:14.489+03:00 INFO 15724 --- [demo] [io-8080-exec-24] c.example.demo.aspect.PerformanceAspect : Finished: ProductService.purchaseWithoutLock | Time: 1406ms
2026-05-15T13:54:14.492+03:00 INFO 15724 --- [demo] [io-8080-exec-20] c.example.demo.aspect.PerformanceAspect : [PURCHASE] Finished: purchaseWithoutLock | Time: 1368ms | Thread: http-nio-8080-exec-20
2026-05-15T13:54:14.549+03:00 INFO 15724 --- [demo] [io-8080-exec-20] c.example.demo.aspect.PerformanceAspect : Finished: ProductService.purchaseWithoutLock | Time: 1467ms
```

من الصورة نستخرج :

exec-24: Finished | Time: 1190ms

exec-9: Finished | Time: 1224ms

exec-4: Finished | Time: 1263ms

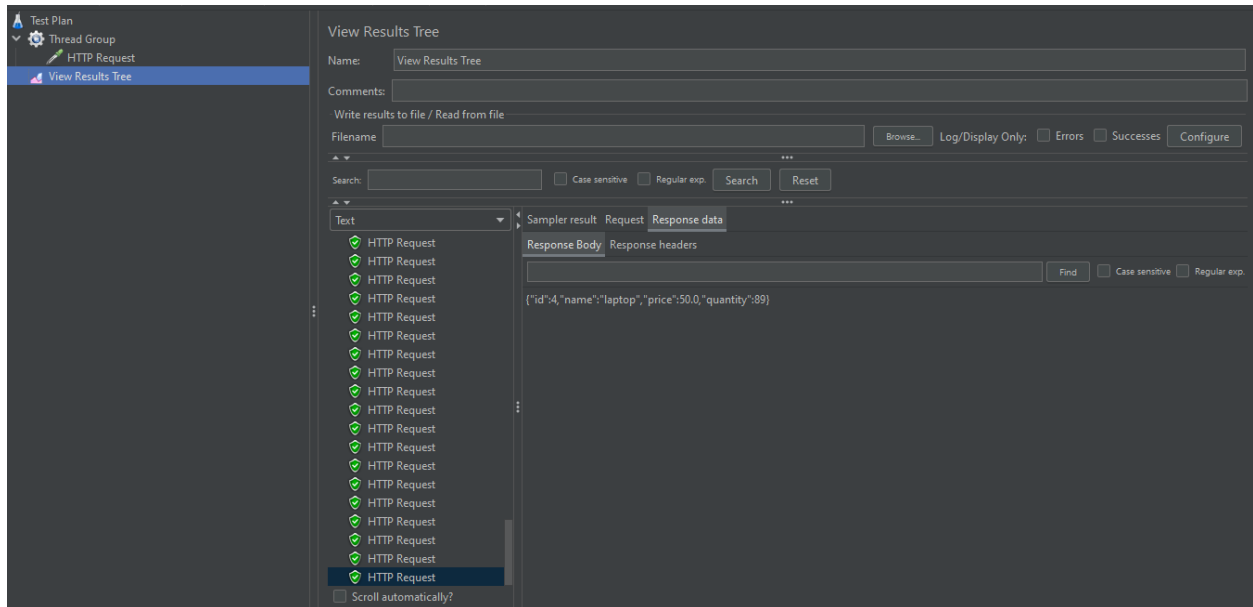
exec-2: Finished | Time: 1261ms

يعني كل ال Threads خلصو بنفس الوقت تقريبا هاد دليل انهم اشتغلو بالتوازي بدون انتظار .

صورة 2 :

صورة من ال Jmeter

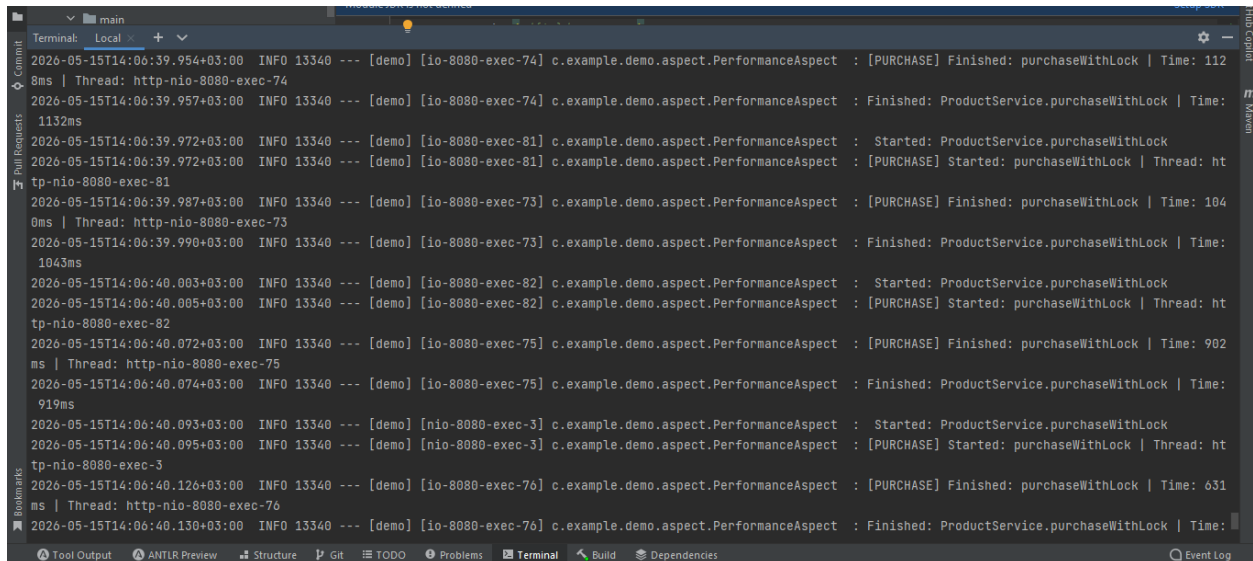
المنتج له 100 كمية اصبحت في اخر طلب 89 وهذا الشيء خاطئ .



بعد التحسين :

صورة 1 :

باستخدام مفهوم الـ AOP(LOG)



من الصورة نستخرج :

exec-74: Finished | Time: 1128ms

exec-73: Finished | Time: 1040ms

exec-75: Finished | Time: 902ms

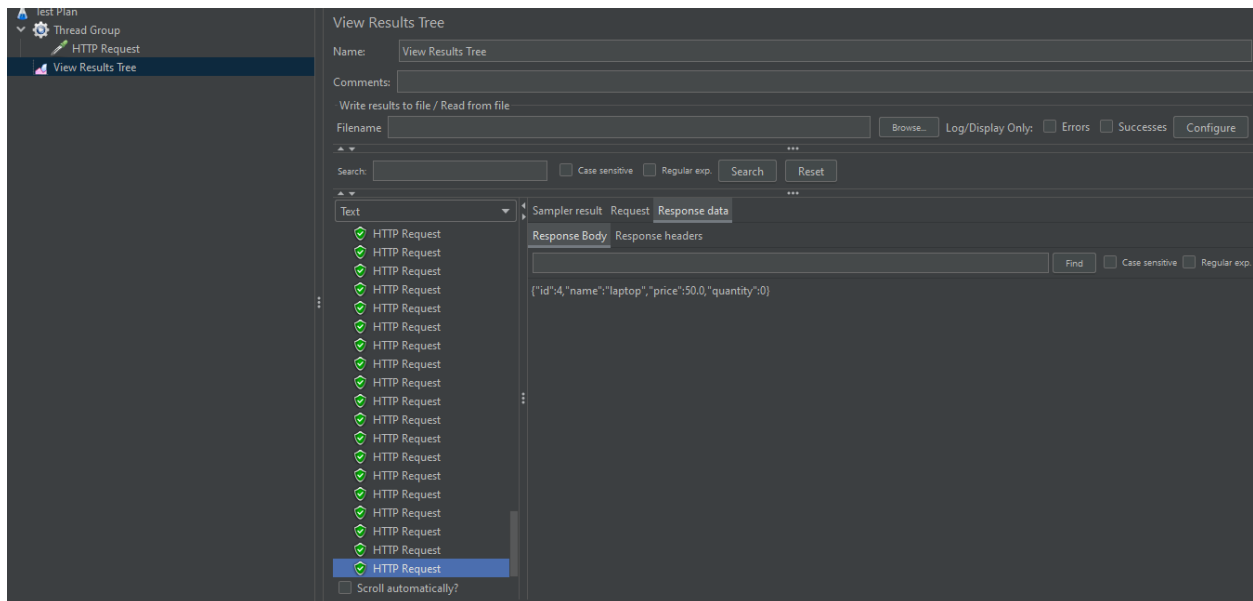
exec-76: Finished | Time: 631ms

نلاحظ ان لكل thread وقت مختلف يعني ان كل thread انتظر ال lock يتحرر قبل ما يشتغل مو كلهم بنفس الوقت .

صورة 2 :

صورة من ال Jmeter

بعد التحسين واطافة ال thread safety اصبحت الكمية 0 وهذا الحل الصحيح



الطلب الثاني :

المشكلة الأساسية :

في الأنظمة التي تستقبل عدد كبير من الطلبات المتزامنة، لا تكمن المشكلة فقط في تنفيذ العمليات بالتوازي، وإنما في كيفية إدارة الموارد الحاسوبية بكفاءة دون استنزاف النظام.

في مشروعنا كانت عملية الشراء (Purchase Operation) تتضمن عدّة خطوات :

- قراءة المنتج من قاعدة البيانات
- قفل المنتج لحماية البيانات
- تحديث الكمية
- حفظ التعديلات (أي حفظ البيانات)
- انتظار زمني (Processing Time) لمحاكاة العمليات الواقعية وقد طبقنا هذا التأخير من خلال استخدام :

○ Thread.sleep(1500)

وهذا يعني أن كل عملية شراء تبقى قيد التنفيذ لفترة زمنية قبل أن تنتهي .

لماذا كانت هذه مشكلة :

لأن نموذج One Thread per Task غير قابل للتوسع عند وجود عدد كبير من الطلبات المتزامنة وذلك لأن كل طلب يحتاج إنشاء Thread مستقل .

ومع ارتفاع عدد المستخدمين المتزامنين يبدأ النظام بإنشاء عدد كبير من الـ Threads وهذا يؤدي إلى:

- استهلاك كبير للذاكرة
- ضغط مرتفع على CPU
- زيادة Context Switching
- انخفاض الأداء
- بطء الاستجابة
- قابلية التوسع محدودة

ومع الضغط العالي قد يصل النظام إلى: Thread Exhaustion أي عدم القدرة على إنشاء Threads إضافية

الهدف من هذا الطلب هو التحكم بعدد العمليات المتوازية وتحسين إدارة الموارد الحاسوبية بحيث يستطيع النظام :

- تحمل عدد كبير من المستخدمين
- منع استنزاف الموارد
- الحفاظ على استقرار الأداء
- زيادة قابلية التوسع
- تحسين استجابة النظام تحت الضغط العالي

الحلول المقترحة :

قبل اختيار الحل النهائي قمنا بدراسة عدّة طرق مستخدمة لإدارة الـ Concurrency والموارد.

أولاً : Traditional Threads

- الحل الأبسط كان الاعتماد على الـ Threads التقليدية
- أي أن كل Request يتم تنفيذه داخل Thread مستقل (أي إنشاء Thread جديد لكل Request)

لماذا لم نستخدم Traditional Threads ؟

- لأن إنشاء عدد كبير من الـ Threads يؤدي إلى مشاكل كبيرة التي ذكرناها سابقاً (غير مناسب للتعامل مع عدد كبير من المستخدمين)
- كما أن كل Thread يعتبر: Heavyweight Thread
- أي أنها:
- مرتبطة مباشرة بنظام التشغيل
- تستهلك Memory كبيرة نسبياً
- إنشاؤها مكلف
- تسبب Context Switching مرتفع
- ومع زيادة عدد الطلبات يبدأ الأداء بالانخفاض بشكل واضح

ثانياً : Rate Limiting

وهو أسلوب يُستخدم لتحديد عدد الطلبات المسموح بها خلال فترة زمنية معينة. فكرته الأساسية:

- منع المستخدم من إرسال Requests كثيرة جداً
- تقليل الضغط على النظام
- حماية السيرفر من الـ Spam أو الـ Abuse

مثال:

- كل مستخدم يُسمح له بـ 100 Request بالدقيقة
- وأي طلبات إضافية يتم:
- رفضها
- أو تأخيرها

لماذا لم يكن مناسباً كمسكلة أساسية عندنا؟

لأن المشكلة التي كنا نحاول حلها لم تكن: تقليل عدد الطلبات القادمة للنظام بل كانت:

- كيفية إدارة وتنفيذ عدد كبير من العمليات المتزامنة بكفاءة
- يعني نحن أردنا أن:
- نستوعب عدد كبير من الطلبات
- وليس منعها

ثالثاً: Thread Pool

والذي يعتمد على إعادة استخدام مجموعة محددة من الـ Threads بدل إنشاء Thread جديد لكل Request

أي الفكرة الأساسية من هذه الطريقة هي :

- إنشاء عدد ثابت من الـ Worker Threads
 - إعادة استخدامهم لتنفيذ عدة Tasks
 - وضع الطلبات الزائدة داخل Queue
- وبالتالي يمنع النظام من إنشاء عدد غير محدود من الـ Threads
- لماذا يعتبر Thread Pool حلاً جيداً؟**
- لأنه:

- يقلل تكلفة إنشاء Threads
- يمنع استنزاف الموارد
- يحسّن الاستقرار
- يجعل استهلاك النظام واقعي

حيث أن هذا الحل يُعتبر Safety Valve كما تحدثنا في المحاضرة

لماذا لم نعتمد Thread Pool كحل نهائي؟

رغم أن Thread Pool يحسّن إدارة الموارد بشكل واضح، إلا أن هناك مشكلة بقيت موجودة وهي أن الـ Thread Pool ما يزال يعتمد على: Platform Threads أي Threads حقيقية من نظام التشغيل.

وهذا يعني أن:

- كل Thread ما يزال يستهلك Memory
 - عدد الـ Threads يبقى محدود
 - عمليات الـ Blocking تبقى الـ Thread محجوز
- وفي مشروعنا توجد عمليات انتظار كثيرة مثل:

• Database Access

• Locks

• Thread.sleep

أي أن أغلب الوقت تكون الـ Threads بحالة Waiting وليس Processing فعلي وبالتالي بقيت المشكلة الأساسية موجودة عند الضغط العالي.

كما أن زيادة حجم الـ Pool بشكل كبير قد تؤدي إلى:

- استهلاك موارد إضافية
- ضغط أعلى على النظام
- زيادة الـ Context Switching

لذلك اعتبرنا أن Thread Pool يحسن الأداء جزئياً لكنه لا يقدم حلاً جذرياً.

قرارنا النهائي :

بعد دراسة الحلول السابقة قررنا استخدام: **Virtual Threads**

وذلك لأنها تقدم طريقة أحدث وأكثر كفاءة لإدارة العمليات المتزامنة

سبب الاختيار :

لأن المشكلة الأساسية في مشروعنا كانت : تشغيل عدد كبير من العمليات المتزامنة دون استنزاف الموارد الحاسوبية وبناءً على هذه المشكلة ظهر لدينا هذا الحل

كيف تعمل Virtual Threads ؟

- بدل إنشاء OS Thread حقيقي لكل Request :
- يتم إنشاء Virtual Thread خفيف جداً
- تقوم JVM بإدارته داخلياً
- يتم تشغيل آلاف Virtual Threads فوق عدد صغير من Carrier Threads

لماذا هذا الحل مناسب جداً لمشروعنا؟
لأن مشروعنا يحتوي عمليات كثيرة من نوع: I/O Bound Operations
مثل:

- الوصول لقاعدة البيانات
 - الانتظار
 - الـ Locks
 - عمليات الـ Sleep
- وكما تحدثنا في المحاضرة أن العمليات الـ I/O Bound تحتاج عدد Threads أكبر من عدد أنوية المعالج بسبب فترات الانتظار الطويلة .
لكن باستخدام Virtual Threads أصبح بالإمكان تشغيل عدد ضخم من العمليات بدون الحاجة لإنشاء عدد ضخم من OS Threads
ماذا يحدث أثناء الانتظار؟
- في الـ Traditional Threads :

- يبقى الـ Thread الحقيقي محجوزاً بالكامل أثناء الانتظار
- أما في Virtual Threads :

تقوم JVM بتحرير الـ Carrier Thread مؤقتاً لخدمة عمليات أخرى وهذا يؤدي إلى:

- تحسين استغلال CPU
- تقليل استهلاك Memory
- تقليل عدد الـ Threads الحقيقية
- رفع قدرة النظام على تحمل الضغط

كيف قمنا بتطبيق الحل داخل المشروع؟
قمنا بإنشاء مسارين للتنفيذ

الوظيفة	العملية
تنفيذ تقليدي	purchaseWithoutVirtual
تنفيذ باستخدام Virtual Threads	purchaseWithVirtual

- لقد قمنا باستخدام @Async ولكن لماذا ؟**
للسماح بتنفيذ العمليات بشكل غير متزامن وهذا أدى إلى:
- تنفيذ عدّة عمليات بالتوازي
 - عدم حجز الـ Main Request Thread

- تحسين استجابة النظام

أيضاً استخدمنا **CompletableFuture** :

لإدارة النتائج الناتجة عن العمليات غير المتزامنة وهذا سمح للنظام بـ:

- Async Processing
- تحسين الـ Throughput
- تقليل الـ Blocking
- إدارة أفضل للطلبات المتزامنة

كيف حافظنا على سلامة البيانات؟

رغم تحسين إدارة الموارد بقيت مشكلة: Race Condition موجودة عند محاولة عدة مستخدمين شراء نفس المنتج.

لذلك استخدمنا: Pessimistic Locking التي استخدمناها لحل الطلب الأول وذلك لضمان:

- حماية البيانات
- منع التضارب
- الحفاظ على Data Consistency

ماذا حققنا ؟

- إدارة أفضل للموارد الحاسوبية
- تقليل استهلاك الـ Memory
- تحسين استغلال الـ CPU
- تشغيل عدد كبير من العمليات المتزامنة
- منع استنزاف النظام تحت الضغط العالي
- تحسين الـ Scalability
- تحسين استقرار النظام
- دعم العمليات الـ I/O Bound بكفاءة أعلى

لماذا اعتبرنا هذا القرار الهندسي مناسب أكثر من باقي الحلول؟

لأن الحل جمع بين:

- إدارة الـ Concurrency
- التحكم بالموارد
- تحسين الأداء
- دعم العمليات الـ Blocking
- الحفاظ على سلامة البيانات

وبالتالي حصلنا على نظام:

- أكثر كفاءة
- أكثر استقراراً
- أكثر قابلية للتوسع
- أقرب للأنظمة الحديثة الواقعية

الاختبار قبل وبعد التحسين:

قبل التحسين :

The screenshot displays the IntelliJ IDEA IDE with a Java project named 'ecommerce_backend1'. The terminal window shows the following log output:

```

load-balancer | 2026-05-18T19:20:59.980Z INFO 1 --- [demo] [cat-handler-109] c.example.demo.aspect.PerformanceAspect : [PURCHASE] Finished: purchaseWithoutVirtual | Time: 15087ms |
Thread: tomcat-handler-109
load-balancer | 2026-05-18T19:20:59.901Z INFO 1 --- [demo] [cat-handler-109] c.example.demo.aspect.PerformanceAspect : Finished: ProductService.purchaseWithoutVirtual | Time: 15088ms |
load-balancer | 2026-05-18T19:21:01.409Z INFO 1 --- [demo] [cat-handler-110] c.example.demo.aspect.PerformanceAspect : [PURCHASE] Finished: purchaseWithoutVirtual | Time: 15085ms |
Thread: tomcat-handler-110
load-balancer | 2026-05-18T19:21:01.409Z INFO 1 --- [demo] [cat-handler-110] c.example.demo.aspect.PerformanceAspect : Finished: ProductService.purchaseWithoutVirtual | Time: 15085ms |
load-balancer | 2026-05-18T19:21:02.918Z INFO 1 --- [demo] [cat-handler-111] c.example.demo.aspect.PerformanceAspect : [PURCHASE] Finished: purchaseWithoutVirtual | Time: 15082ms |
Thread: tomcat-handler-111
load-balancer | 2026-05-18T19:21:02.919Z INFO 1 --- [demo] [cat-handler-111] c.example.demo.aspect.PerformanceAspect : Finished: ProductService.purchaseWithoutVirtual | Time: 15083ms |
load-balancer | 2026-05-18T19:21:04.429Z INFO 1 --- [demo] [cat-handler-112] c.example.demo.aspect.PerformanceAspect : [PURCHASE] Finished: purchaseWithoutVirtual | Time: 15083ms |
Thread: tomcat-handler-112
load-balancer | 2026-05-18T19:21:04.429Z INFO 1 --- [demo] [cat-handler-112] c.example.demo.aspect.PerformanceAspect : Finished: ProductService.purchaseWithoutVirtual | Time: 15083ms |
load-balancer | 2026-05-18T19:21:05.943Z INFO 1 --- [demo] [cat-handler-113] c.example.demo.aspect.PerformanceAspect : [PURCHASE] Finished: purchaseWithoutVirtual | Time: 15089ms |
Thread: tomcat-handler-113
load-balancer | 2026-05-18T19:21:05.943Z INFO 1 --- [demo] [cat-handler-113] c.example.demo.aspect.PerformanceAspect : Finished: ProductService.purchaseWithoutVirtual | Time: 15089ms |
load-balancer | 2026-05-18T19:21:07.452Z INFO 1 --- [demo] [cat-handler-114] c.example.demo.aspect.PerformanceAspect : [PURCHASE] Finished: purchaseWithoutVirtual | Time: 15087ms |
Thread: tomcat-handler-114
load-balancer | 2026-05-18T19:21:07.452Z INFO 1 --- [demo] [cat-handler-114] c.example.demo.aspect.PerformanceAspect : Finished: ProductService.purchaseWithoutVirtual | Time: 15087ms |
load-balancer | 2026-05-18T19:21:08.960Z INFO 1 --- [demo] [cat-handler-115] c.example.demo.aspect.PerformanceAspect : [PURCHASE] Finished: purchaseWithoutVirtual | Time: 15082ms |
Thread: tomcat-handler-115
load-balancer | 2026-05-18T19:21:08.960Z INFO 1 --- [demo] [cat-handler-115] c.example.demo.aspect.PerformanceAspect : Finished: ProductService.purchaseWithoutVirtual | Time: 15082ms |
load-balancer | 2026-05-18T19:21:10.472Z INFO 1 --- [demo] [cat-handler-116] c.example.demo.aspect.PerformanceAspect : [PURCHASE] Finished: purchaseWithoutVirtual | Time: 15086ms |

```

حيث أن كل طلب يقوم tomcat بإنشاء ثريد خاص كما موضح بالصورة

without_virtual.jmx (C:\IntelliJ\ecommerce_backend\src\main\java\example\demo\meter\without_virtual.jmx) - Apache JMeter (5.6.3)

File Edit Search Run Options Tools Help

Test Plan

Thread Group

HTTP Request

View Results Tree

Summary Report

View Results in Table

View Results in Table

Name: View Results in Table

Comments:

Write results to file / Read from file

Filename

Browse...

Log/Display Only: ☐ Errors ☐ Successes ☐ Configure

Sample #	Start Time	Thread Name	Label	Sample Time(ms)	Status	Bytes	Sent Bytes	Latency	Connect Time(ms)
1	22-20:40.266	Thread Group 1-1	HTTP Request	1525	OK	211	237	1525	2
2	22-20:40.316	Thread Group 1-2	HTTP Request	2987	OK	211	237	2987	1
3	22-20:40.365	Thread Group 1-3	HTTP Request	4448	OK	211	237	4448	0
4	22-20:40.416	Thread Group 1-4	HTTP Request	5908	OK	211	237	5908	1
5	22-20:40.465	Thread Group 1-5	HTTP Request	7370	OK	211	237	7370	0
6	22-20:40.515	Thread Group 1-6	HTTP Request	8830	OK	211	237	8830	1
7	22-20:40.566	Thread Group 1-7	HTTP Request	10287	OK	211	237	10287	1
8	22-20:40.616	Thread Group 1-8	HTTP Request	11749	OK	211	237	11749	1
9	22-20:40.665	Thread Group 1-9	HTTP Request	13212	OK	211	237	13212	1
10	22-20:40.715	Thread Group 1-10	HTTP Request	14670	OK	211	237	14670	0
11	22-20:40.765	Thread Group 1-11	HTTP Request	16128	OK	211	237	16128	1
12	22-20:40.817	Thread Group 1-12	HTTP Request	17586	OK	211	237	17586	1
13	22-20:40.865	Thread Group 1-13	HTTP Request	19046	OK	211	237	19046	1
14	22-20:40.916	Thread Group 1-14	HTTP Request	20504	OK	211	237	20504	1
15	22-20:40.965	Thread Group 1-15	HTTP Request	21965	OK	211	237	21965	1
16	22-20:41.015	Thread Group 1-16	HTTP Request	23429	OK	211	237	23429	1
17	22-20:41.064	Thread Group 1-17	HTTP Request	24890	OK	211	237	24890	1
18	22-20:41.115	Thread Group 1-18	HTTP Request	26347	OK	211	237	26347	1
19	22-20:41.164	Thread Group 1-19	HTTP Request	27809	OK	211	237	27809	1
20	22-20:41.214	Thread Group 1-20	HTTP Request	29268	OK	211	237	29268	1

☐ Scroll automatically?

☐ Child samples?

No of Samples 20

Latest Sample 29268

View Log (1500)

Download CSV

without_virtualjmx (C:\intelli\ecommerce_backend1\src\main\java\com\example\demo\meter_files\without_virtualjmx) - Apache JMeter (5.6.3)

File Edit Search Run Options Tools Help

00:00:30 0 0/20

Test Plan

- Thread Group
 - HTTP Request
 - View Results Tree
 - Summary Report
 - View Results in Table

Summary Report

Name: Summary Report

Comments:

Write results to file / Read from file

Filename: Log/Display Only: ☐ Errors ☐ Successes

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
HTTP Request	20	15397	1525	29268	8418.73	0.00%	39.7/min	0.14	0.15	211.0
TOTAL	20	15397	1525	29268	8418.73	0.00%	39.7/min	0.14	0.15	211.0

Include group name in label? ☐ Save Table Data Save Table Header ☒

without_virtualjmx (C:\intelli\ecommerce_backend1\src\main\java\com\example\demo\meter_files\without_virtualjmx) - Apache JMeter (5.6.3)

File Edit Search Run Options Tools Help

00:00:30 0 0/20

Test Plan

- Thread Group
 - HTTP Request
 - View Results Tree
 - Summary Report
 - View Results in Table

View Results Tree

Name: View Results Tree

Comments:

Write results to file / Read from file

Filename: Log/Display Only: ☐ Errors ☐ Successes

Search: Case sensitive ☐ Regular exp. ☐

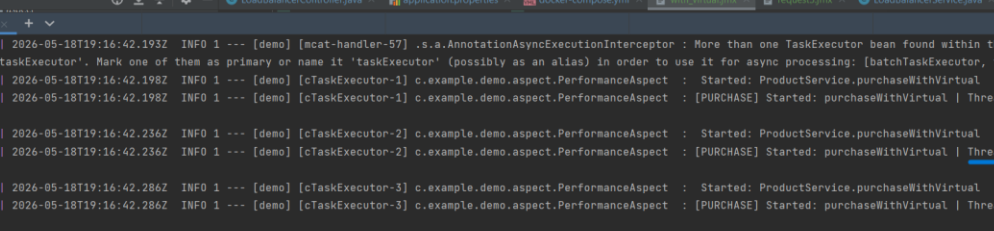
Text

Sampler result Request Response data

- HTTP Request
 - Thread Name: Thread Group 1-1
 - Sample Start: 2026-05-18 22:20:40 GMT+03:00
 - Load time: 1525
 - Connect Time: 2
 - Latency: 1525
 - Size in bytes: 211
 - Sent bytes: 237
 - Headers size in bytes: 162
 - Body size in bytes: 49
 - Sample Count: 1
 - Error Count: 0
 - Data type ("text/plain"): text
 - Response code: 200
 - Response message:
- HTTP Request
 - HTTPSampleResult fields:
 - ContentType: text/plain; charset=UTF-8
 - DataEncoding: UTF-8

Scroll automatically? ☐ Raw ☒ Parsed

بعد التحسين :



```
File Edit View Navigate Code Refactor Build Run Tools Git Window Help ecommerce_backend1 - with_virtualjmx
ecommerce_backend1 | src | main | java | com | example | demo | jmeter_files | with_virtualjmx
Project | Local (3) | + | - | LoadBalancerController.java | application.properties | docker-compose.yml | with_virtualjmx | request5.jmx | LoadBalancerService.java | BatchCon...
Terminal | Local (3) | + | - |
load-balancer | 2026-05-18T19:16:42.193Z | INFO | 1 | --- [demo] [mcmt-handler-57] s.a.AnnotationAsyncExecutionInterceptor : More than one TaskExecutor bean found within the context, and none is named 'taskExecutor'. Mark one of them as primary or name it 'taskExecutor' (possibly as an alias) in order to use it for async processing: [batchTaskExecutor, taskScheduler]
load-balancer | 2026-05-18T19:16:42.198Z | INFO | 1 | --- [demo] [cTaskExecutor-1] c.example.demo.aspect.PerformanceAspect : Started: ProductService.purchaseWithVirtual
load-balancer | 2026-05-18T19:16:42.198Z | INFO | 1 | --- [demo] [cTaskExecutor-1] c.example.demo.aspect.PerformanceAspect : [PURCHASE] Started: purchaseWithVirtual | Thread: SimpleAsyncT
askExecutor-1 | 2026-05-18T19:16:42.236Z | INFO | 1 | --- [demo] [cTaskExecutor-2] c.example.demo.aspect.PerformanceAspect : Started: ProductService.purchaseWithVirtual
load-balancer | 2026-05-18T19:16:42.236Z | INFO | 1 | --- [demo] [cTaskExecutor-2] c.example.demo.aspect.PerformanceAspect : [PURCHASE] Started: purchaseWithVirtual | Thread: SimpleAsyncT
askExecutor-2 | 2026-05-18T19:16:42.286Z | INFO | 1 | --- [demo] [cTaskExecutor-3] c.example.demo.aspect.PerformanceAspect : Started: ProductService.purchaseWithVirtual
load-balancer | 2026-05-18T19:16:42.286Z | INFO | 1 | --- [demo] [cTaskExecutor-3] c.example.demo.aspect.PerformanceAspect : [PURCHASE] Started: purchaseWithVirtual | Thread: SimpleAsyncT
askExecutor-3 | 2026-05-18T19:16:42.336Z | INFO | 1 | --- [demo] [cTaskExecutor-4] c.example.demo.aspect.PerformanceAspect : Started: ProductService.purchaseWithVirtual
load-balancer | 2026-05-18T19:16:42.336Z | INFO | 1 | --- [demo] [cTaskExecutor-4] c.example.demo.aspect.PerformanceAspect : [PURCHASE] Started: purchaseWithVirtual | Thread: SimpleAsyncT
askExecutor-4 | 2026-05-18T19:16:42.387Z | INFO | 1 | --- [demo] [cTaskExecutor-5] c.example.demo.aspect.PerformanceAspect : Started: ProductService.purchaseWithVirtual
load-balancer | 2026-05-18T19:16:42.388Z | INFO | 1 | --- [demo] [cTaskExecutor-5] c.example.demo.aspect.PerformanceAspect : [PURCHASE] Started: purchaseWithVirtual | Thread: SimpleAsyncT
askExecutor-5 | 2026-05-18T19:16:42.435Z | INFO | 1 | --- [demo] [cTaskExecutor-6] c.example.demo.aspect.PerformanceAspect : Started: ProductService.purchaseWithVirtual
load-balancer | 2026-05-18T19:16:42.435Z | INFO | 1 | --- [demo] [cTaskExecutor-6] c.example.demo.aspect.PerformanceAspect : [PURCHASE] Started: purchaseWithVirtual | Thread: SimpleAsyncT
askExecutor-6 | 2026-05-18T19:16:42.486Z | INFO | 1 | --- [demo] [cTaskExecutor-7] c.example.demo.aspect.PerformanceAspect : Started: ProductService.purchaseWithVirtual
load-balancer | 2026-05-18T19:16:42.486Z | INFO | 1 | --- [demo] [cTaskExecutor-7] c.example.demo.aspect.PerformanceAspect : [PURCHASE] Started: purchaseWithVirtual | Thread: SimpleAsyncT
askExecutor-7 | 2026-05-18T19:16:42.535Z | INFO | 1 | --- [demo] [cTaskExecutor-8] c.example.demo.aspect.PerformanceAspect : Started: ProductService.purchaseWithVirtual
load-balancer | 2026-05-18T19:16:42.536Z | INFO | 1 | --- [demo] [cTaskExecutor-8] c.example.demo.aspect.PerformanceAspect : [PURCHASE] Started: purchaseWithVirtual | Thread: SimpleAsyncT
askExecutor-8 | 2026-05-18T19:16:42.585Z | INFO | 1 | --- [demo] [cTaskExecutor-9] c.example.demo.aspect.PerformanceAspect : Started: ProductService.purchaseWithVirtual
load-balancer | 2026-05-18T19:16:42.585Z | INFO | 1 | --- [demo] [cTaskExecutor-9] c.example.demo.aspect.PerformanceAspect : [PURCHASE] Started: purchaseWithVirtual | Thread: SimpleAsyncT
askExecutor-9 |
Tool Output | ANTLR Preview | Git | TODO | Problems | Terminal | Build | Dependencies | No occurrences found | 1:1 LF UTF-8 4 spaces | Event Log | Pageham
```

The screenshot shows the Apache JMeter interface. On the left, the Test Plan tree is expanded, showing a Thread Group with an HTTP Request. The main window displays the 'View Results Tree' panel. The 'Name' field is 'View Results Tree'. The 'Comments' field is empty. The 'Write results to file / Read from file' section has a 'Filename' field and a 'Browse...' button. The 'Log/Display Only' section has checkboxes for 'Errors' and 'Successes', and a 'Configure' button. The 'Search' section has a 'Search' field and buttons for 'Case sensitive', 'Regular exp.', 'Search', and 'Reset'. The 'Text' section is expanded, showing a list of 'HTTP Request' items. The 'Response data' tab is selected, showing the 'Response Body' field with the text 'SUCCESS: Purchased 1 x Mobile A55 | Remaining: 19'. The 'Find' button and 'Case sensitive' / 'Regular exp.' checkboxes are also visible.

The screenshot shows the Apache JMeter GUI with the 'View Results in Table' window open. The window title is 'with_virtual_jmx (C:\intelli\ecommerce_backend1\src\main\java\example\demo\jmeter_req\with_virtual_jmx) - Apache JMeter (5.6.3)'. The left sidebar shows the test plan structure: Test Plan, Thread Group, HTTP Request, View Results Tree, Summary Report, and View Results in Table (selected). The main window displays the 'View Results in Table' dialog with the following fields: Name (View Results in Table), Comments, Write results to file / Read from file, Filename, Browse... button, Log/Display Only checkbox, Errors checkbox, Successes checkbox, and Configure button. Below these fields is a table of test results.

Sample #	Start Time	Thread Name	Label	Sample Time(ms)	Status	Bytes	Sent Bytes	Latency	Connect Time(ms)
1	23.3035.158	Thread Group 1-1	HTTP Request	1786	Success	211	234	1786	3
2	23.3035.257	Thread Group 1-2	HTTP Request	3171	Success	211	234	3171	0
3	23.3035.358	Thread Group 1-3	HTTP Request	4583	Success	211	234	4583	1
4	23.3035.457	Thread Group 1-4	HTTP Request	5994	Success	211	234	5994	0
5	23.3035.557	Thread Group 1-5	HTTP Request	7404	Success	211	234	7404	1
6	23.3035.657	Thread Group 1-6	HTTP Request	8813	Success	211	234	8813	0
7	23.3035.756	Thread Group 1-7	HTTP Request	10227	Success	211	234	10227	0
8	23.3035.856	Thread Group 1-8	HTTP Request	11636	Success	211	234	11636	0
9	23.3035.956	Thread Group 1-9	HTTP Request	13045	Success	211	234	13045	0
10	23.3036.056	Thread Group 1-10	HTTP Request	14456	Success	211	234	14456	1

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
HTTP Request	10	8111	1786	14436	4047.68	0.00%	39.1/min	0.13	0.15	211.0
TOTAL	10	8111	1786	14436	4047.68	0.00%	39.1/min	0.13	0.15	211.0

الطلب الثالث :

1. المشكلة

في أنظمة التجارة الإلكترونية، عند إتمام عملية الشراء تحدث عدة مهام متسلسلة: تحديث المخزون، معالجة الدفع، إرسال إشعار للعميل، وإصدار فاتورة. في النهج التقليدي المتزامن، يظل المستخدم محجوباً في انتظار اكتمال جميع هذه المهام، بما فيها مهام ثانوية كإرسال البريد الإلكتروني التي لا تتطلب انتظاره. هذا يسبب زمن استجابة مرتفعاً جداً يصل إلى 5-8 ثوانٍ لكل طلب، مما يُضعف تجربة المستخدم ويُقيّد قدرة النظام على خدمة طلبات متعددة في آنٍ واحد.

جوهر المشكلة: كيف نُخرج المهام الثانوية من المسار الرئيسي للطلب بحيث يحصل المستخدم على استجابة فورية، مع ضمان تنفيذ هذه المهام لاحقاً بشكل موثوق؟

مثال

عند إتمام الشراء عملية الشراء النظام كان يقوم بإرسال الإشعارات بشكل متزامن مما يجبر المستخدم على الانتظار حتى تنتهي كل العمليات

مثال :

- مستخدم اشترى منتج
- النظام سجل الطلب
- النظام بدأ يرسل إيميل تأكيد : المستخدم ينتظر 3 ثواني
- النظام اصدر فاتورة : المستخدم لا يزال ينتظر

النتيجة : تجربة مستخدم سيئة وبطيئة في الاستجابة .

2.الحلول المقترحة

الحل الاول synchronous(بدون تحسين)

في هذا النهج تنفذ جميع المهام تسلسلياً في نفس Thread الرئيسي قبل الرد على المستخدم. يضمن هذا الحل البساطة والوضوح، لكنه يعاني من مشكلة جوهرية : المستخدم ينتظر 3 ثوانٍ أو أكثر لمجرد تأكيد الدفع، وإن فشل إرسال الإشعار فشل الطلب كله معه. تم استبعاد هذا الحل لأنه غير مقبول في بيئة إنتاجية تتعامل مع آلاف الطلبات المتزامنة.

الحل الثاني Async

يعتمد هذا الحل على تشغيل المهام الثانوية في Threads منفصلة داخل نفس التطبيق باستخدام Spring. @Async في Spring يحسن زمن الاستجابة بشكل ملحوظ، لكنه يحمل عيباً خطيراً : المهام تخزن في ذاكرة الـ JVM فقط، فإذا أعيد تشغيل الخادم أو انهار فجأة، تفقد جميع المهام المعلقة نهائياً دون أي إمكانية للاسترداد. هذا غير مقبول في سياق معالجة مدفوعات.

الحل الثالث Message Queue

يعتمد على إرسال المهام الثانوية كرسائل إلى قائمة انتظار مستدامة (Durable Queue) يعالجها Worker مستقل في الخلفية. هذا الحل يجمع بين السرعة في الاستجابة وضمان تنفيذ المهام، وهو المعيار الصناعي لهذا النوع من المشاكل.

3. سبب اختيار الـ Async

اختير RabbitMQ لأسباب تقنية محددة

أولاً:

الرسائل محفوظة على القرص (Durable Queues) ولا تفقد عند إعادة تشغيل الخادم.

ثانياً:

يعتمد على نظام التأكيد (ACK/NACK) ، بمعنى أن المستهلك يرسل تأكيداً للرسالة فقط بعد معالجتها بنجاح، وإن حدث خطأ تعود الرسالة تلقائياً للقائمة لإعادة المحاولة.

ثالثاً:

يوفر فصلاً تاماً بين المنتج والمستهلك (Decoupling) ، مما يعني أن فشل خدمة الإشعارات لا يؤثر بأي شكل على إتمام الطلب الأصلي

رابعاً:

يدعم التوسع الأفقي بإضافة consumers متعددين لنفس القائمة لرفع الإنتاجية عند الحاجة.

الآلية التنفيذية :

عند استدعاء (end point) /api/payment/async ، ينفذ المسار الرئيسي السريع فقط: التحقق من الرصيد، خصم المبلغ، حفظه في قاعدة البيانات، ثم إرسال رسالة إلى payment.queue في RabbitMQ. يعود الرد للمستخدم فوراً خلال أقل من 100 ms. في الخلفية وبشكل مستقل تماماً،

وعندها يبدأ الـ Consumer عبر @RabbitListener، يسحب الرسالة، ينفذ المعالجة الفعلية (3) ثوانٍ)، ويُرسَل تأكيد ACK عند النجاح.

تم تعريف قائمتين منفصلتين في RabbitMQConfig:

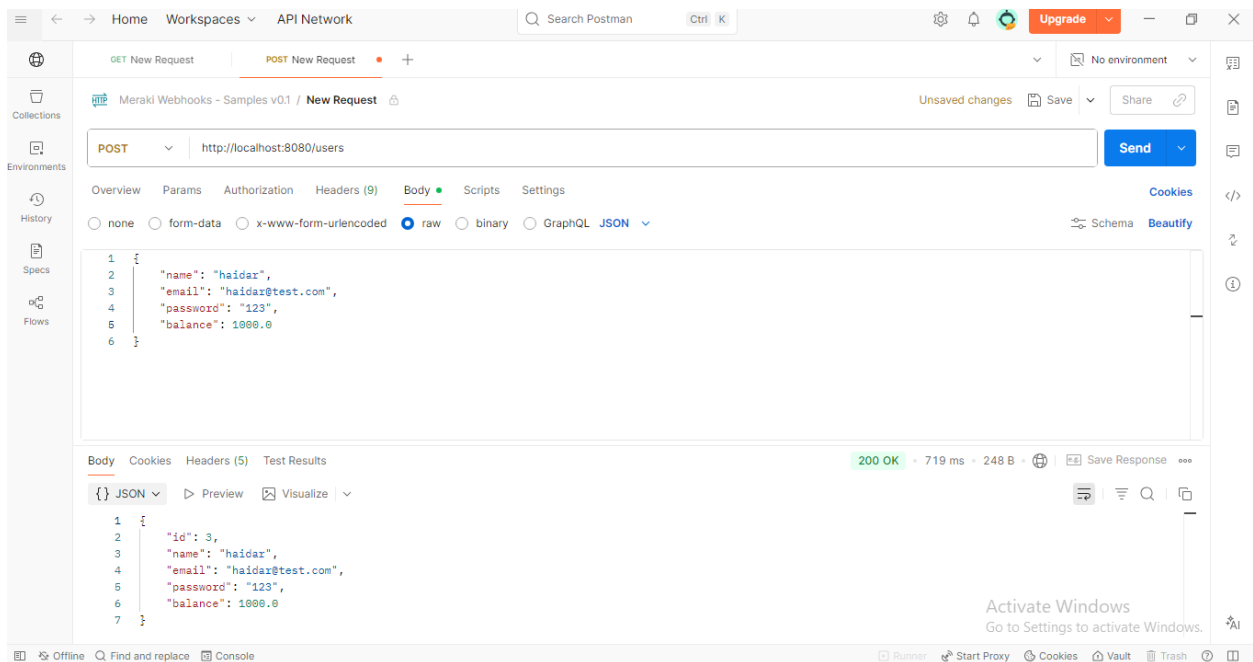
- **payment.queue** : لمعالجة تأكيدات الدفع
- **notification.queue** : لإرسال الإشعارات والفواتير

4. نتائج الاختبار بـ postman

سنجرب في هذا الاختبار نظام الدفع :

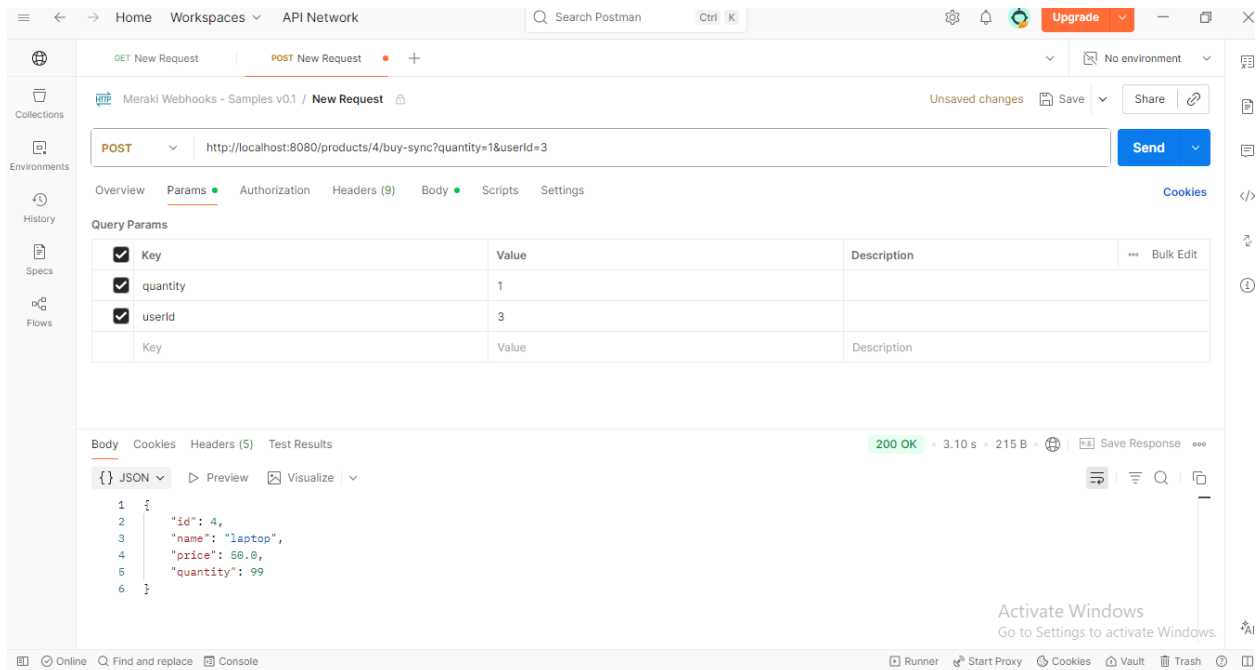
اولا :

سنضيف مستخدم جديد

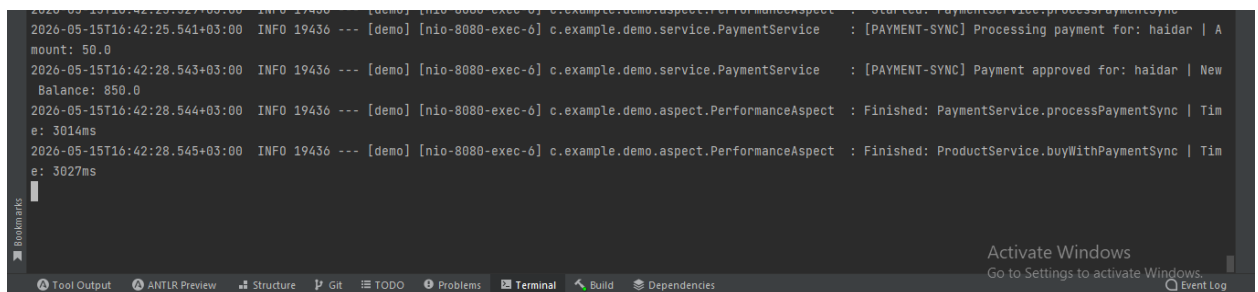


ثانيا :

سنجرب الدفع باستخدام الـ Blocking



قبل التحسين استغرق الامر 3 ثواني



[nio-8080-exec-6] PaymentService : [PAYMENT-SYNC] Processing payment for: haidar | Amount: 50.0

بدأت المعالجة... والمستخدم لا زال ينتظر.

[PAYMENT-SYNC] Payment approved for: haidar | New Balance: 850.0

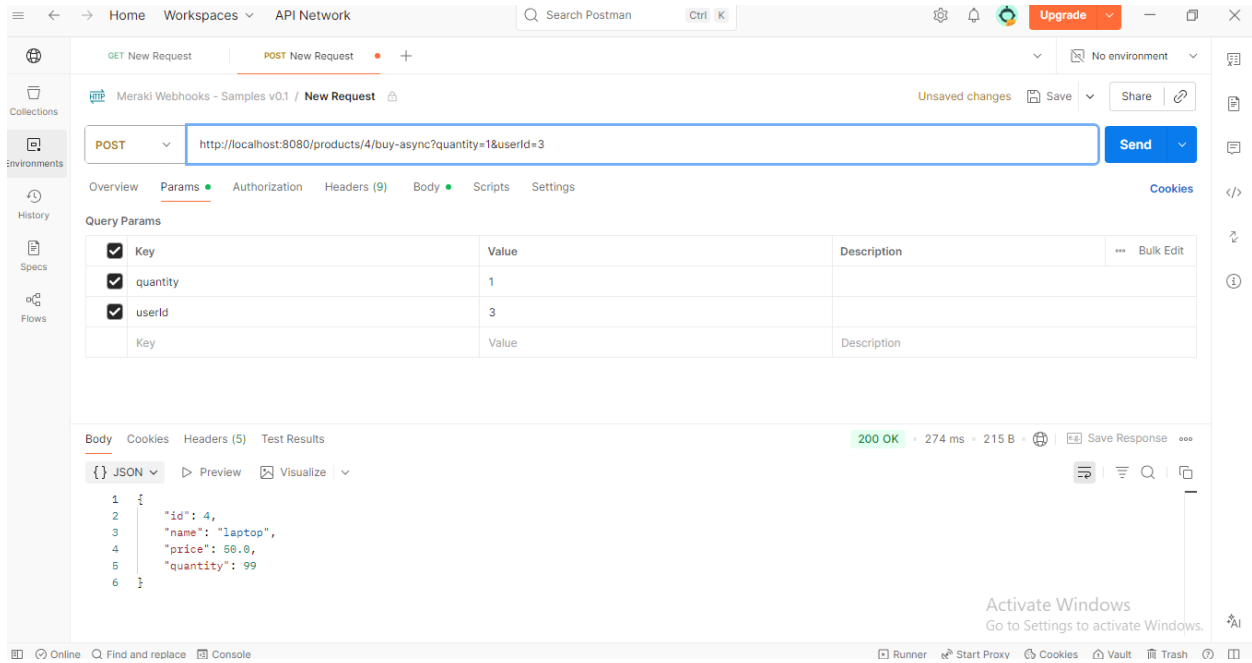
اكتملت المعالجة، الرصيد صار 850.

Finished: ProductService.buyWithPaymentSync | Time: 3027ms

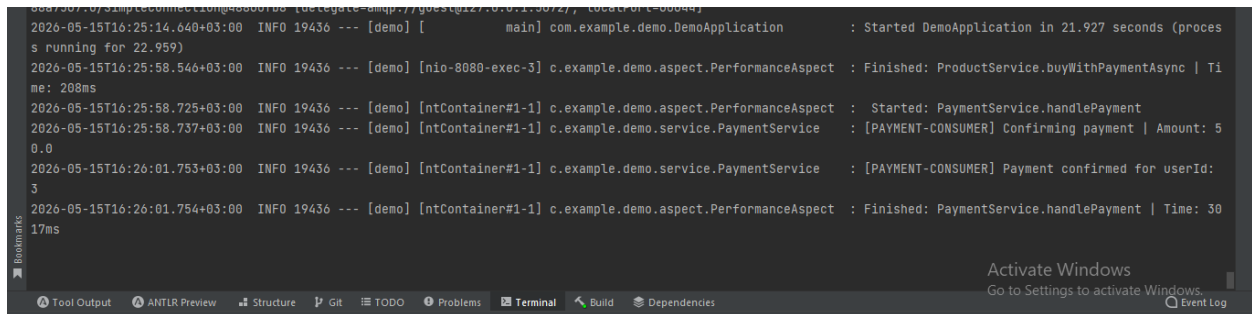
لمستخدم انتظر 3027ms كاملة قبل ما يحصل على الرد.

ثالثا :

سنجرب الدفع الان باستخدام ال non blocking



بعد التحسين استغرق 274ms اي ان الرد كان سريع وهذا يوضح الفرق



هنا صورة من التيرمنال عن ال logs سنشرح شو صار بالضبط :

[nio-8080-exec-3] PerformanceAspect : Finished :
ProductService.buyWithPaymentAsync | Time: 208ms

لصار انو المستخدم طلب الشراء فالطلب اكتمل وعاد للمستخدم خلال 208ms

[ntContainer#1-1] PerformanceAspect : Started:
PaymentService.handlePayment [ntContainer#1-1] PaymentService :
[PAYMENT-CONSUMER] Confirming payment | Amount: 50.0

هنا بدأ ال consumer يشتغل بشكل مستقل في thread مختلف تماماً

[ntContainer#1-1] PaymentService : [PAYMENT-CONSUMER] Payment
confirmed for userId: 3

[ntContainer#1-1] PerformanceAspect : Finished:
PaymentService.handlePayment | Time: 3017ms

ال consumer خلص شغله بعد 3 ثواني يعني (لمعالجة الفعلية أخذت 3017ms لكن المستخدم ما انتظرها) .

الطلب الرابع :

1. المشكلة

في أنظمة التجارة الإلكترونية، تتراكم كميات ضخمة من بيانات المبيعات يومياً. في نهاية كل يوم يحتاج النظام لمعالجة هذه السجلات، كحساب التقارير اليومية وتحديث حالة المبيعات وعند معالجة كميات كبيرة من بيانات المبيعات دفعة واحدة يحدث ما يلي :

- تحميل جميع السجلات لل RAM دفعة واحدة يؤدي الى انهيار النظام .
- معالجة السجلات بشكل تسلسلي تستغرق وقتاً طويلاً جداً .

على سبيل المثال كل سجل يأخذ 1 ثانية ولدينا 10 ملايين سجل المعالجة التسلسلية تأخذ 115 يوم

2.الحلول المقترحة

الحل الاول معالجة كل البيانات دفعة واحدة

يعتمد على جلب جميع السجلات بأمر findAll() ومعالجتها في حلقة واحدة. بسيط في الكتابة لكنه كارثي على الأداء، إذ يحمل كل شيء في الذاكرة (RAM) مرة واحدة. مع 100,000 سجل يستهلك هذا

الحل مئات الميغابايت من الذاكرة، ويجمد قاعدة البيانات لفترة طويلة، ويرفع احتمالية الانهيار الكامل. تم استبعاده لأنه غير قابل للتوسع أبدا .

الحل الثاني Fixed – size chunking

يعتمد على تقسيم البيانات إلى دفعات بحجم ثابت محدد مسبقا، مثلا 100 سجل لكل دفعة بغض النظر عن العدد الكلي.

أبسط في التنفيذ من الحل المختار، لكنه يحمل مشكلة جوهرية: الحجم الثابت لا يتكيف مع تغير البيانات. إن كان عندك 500 سجل اليوم و500,000 سجل غدا، نفس الحجم الثابت يسبب إما كثرة استعلامات مع البيانات القليلة أو بطء مع البيانات الكثيرة. تم استبعاده لصالح الحجم الديناميكي الذي يتكيف تلقائيا مع حجم البيانات الفعلي.

الحل الثالث Dynamic Partitioning + spring Batch (المختار)

يعتمد هذا الحل على **Spring Batch** الذي يوفر بنية جاهزة لمعالجة البيانات الضخمة بطريقة منظمة وموثوقة، مع تقسيم العمل بشكل ذكي إلى أجزاء قابلة للتنفيذ بالتوازي (هو مخصص لمعالجة ال batch processing في ال spring Boot).

- يتم تقسيم البيانات إلى Partitions (نطاقات IDs)
- كل Partition يتم معالجته بواسطة Worker Thread مستقل .
- داخل كل Worker يتم استخدام Chunk-oriented Processing

3.سبب اختيار Dynamic Partitioning

1. حجم Chunk ديناميكي (Dynamic Chunk Size) :

يتم حساب حجم الدفعة حسب عدد السجلات الفعلي:

- حتى $10 \leq 100$
- حتى $100 \leq 1000$
- حتى $500 \leq 10,000$
- أكثر من ذلك $1000 \leq$

2. تقليل الضغط على الذاكرة (Memory Efficient) :

Spring Batch لا يحمل كل البيانات دفعة واحدة، بل يعالج كل Chunk بشكل مستقل

(Read → Process → Write)

3. تحسين الأداء عبر التوازي (Parallel Processing) :

باستخدام TaskExecutorPartition يتم تشغيل عدة Workers بالتوازي، مما يقلل وقت التنفيذ بشكل كبير.

4. إدارة الحالة (Job State Management) :

Spring Batch يقوم بتخزين حالة الـ Job في قاعدة البيانات عبر:

- JobRepository
- JobExplorer

وهذا يسمح باستكمال التنفيذ بعد الانقطاع (Restartability) .

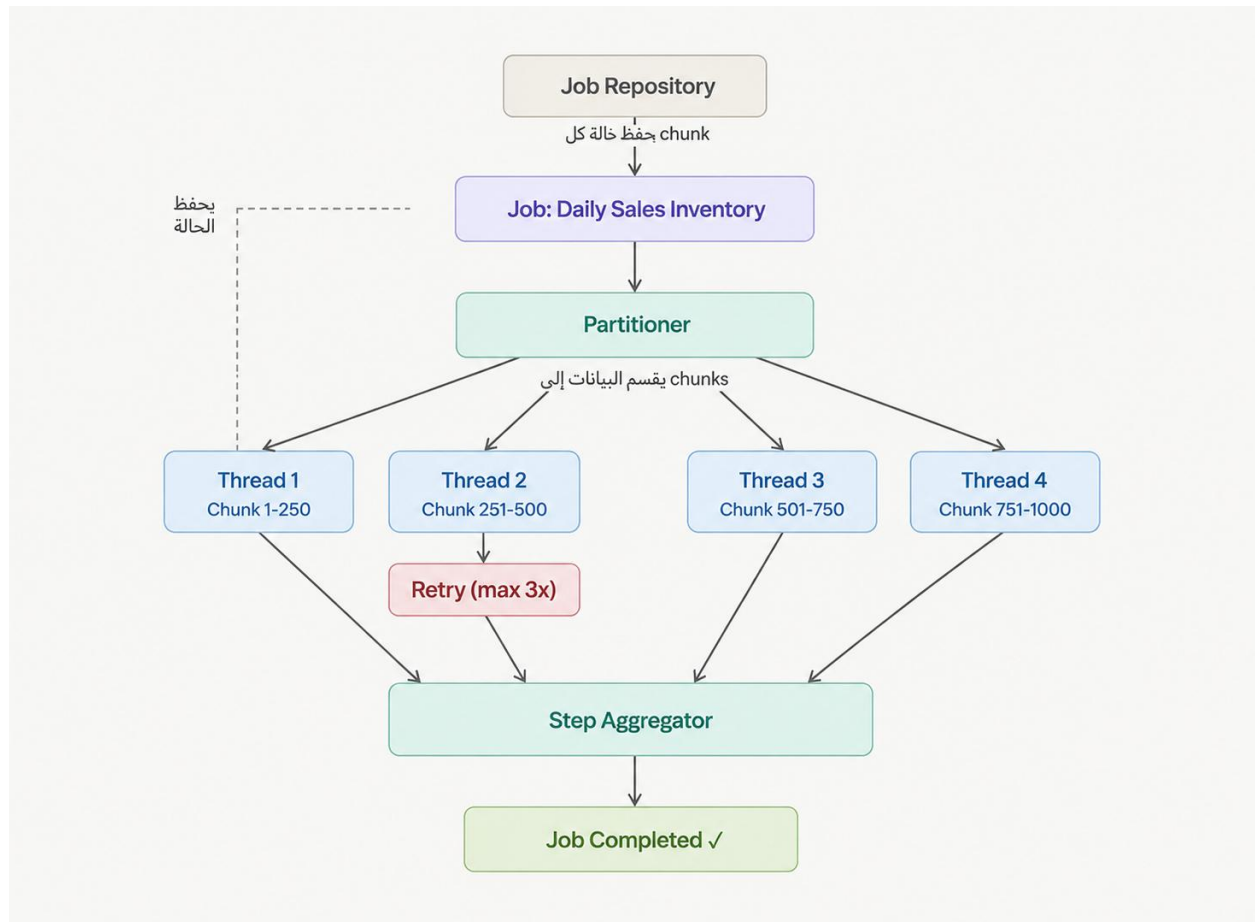
5. Fault Tolerance

الـ Fault Tolerance ييحفظ حالة كل step في الـ DB جدول إذا مات الـ container في المنتصف، لما يرجع بيعرف من وين يكمل

يدعم:

- retry لإعادة محاولة معالجة السجل عند الفشل
- skip لتجاوز السجلات الخاطئة دون إيقاف العملية كاملة

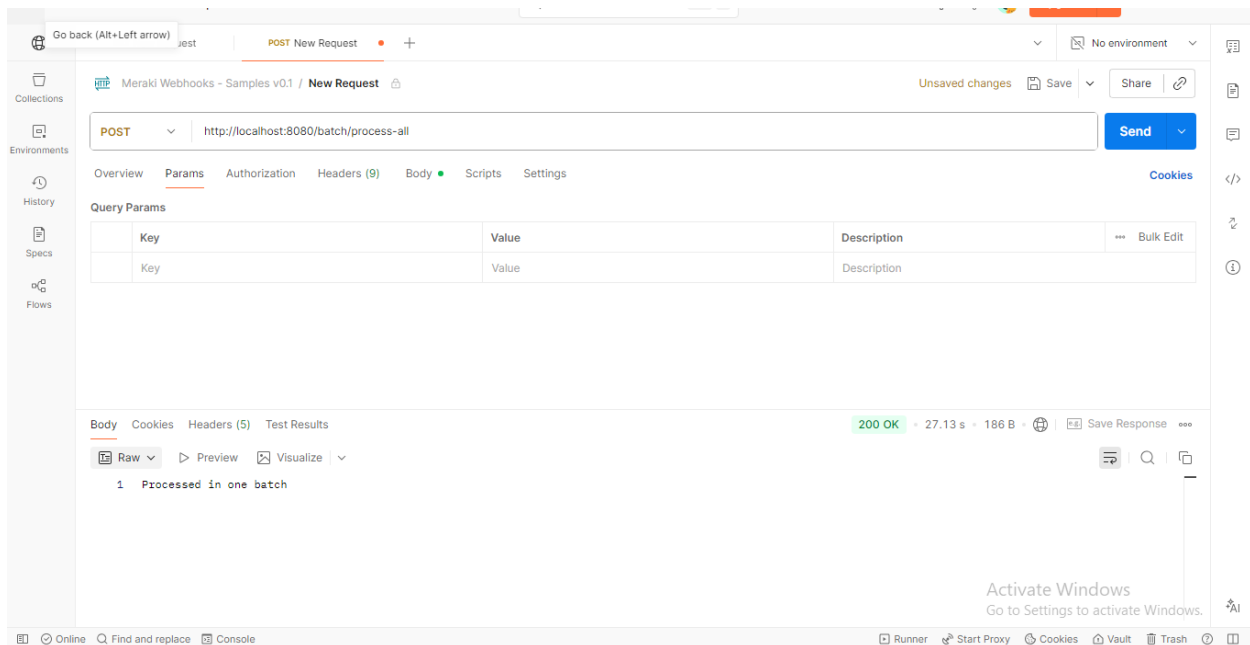
6. هذا Diagram يوضح طريقة عملها :



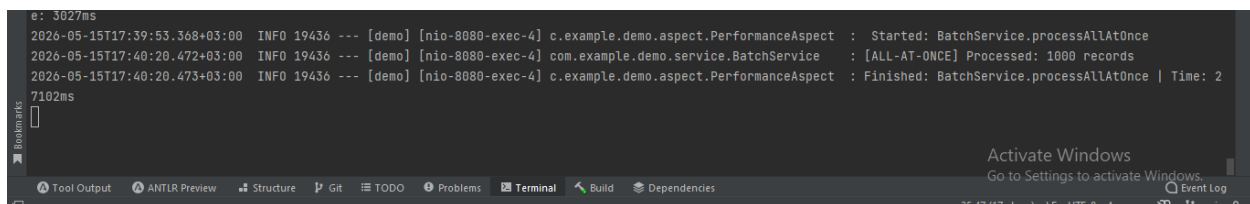
4. نتائج الاختبار بـ postman

تم اختبار النظام في postman على 1000 records:

اولا : قبل التحسين



اسغرق الأمر 27 ثانية

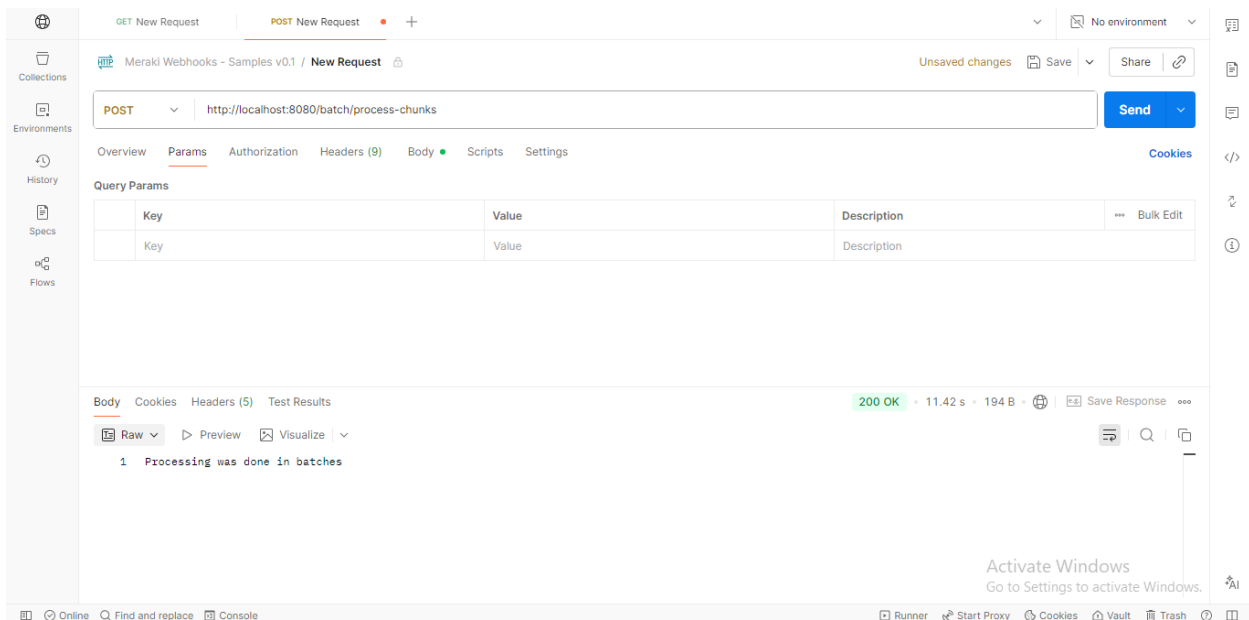


Started: BatchService.processAllAtOnce
 هذا يعني أن النظام بدأ تشغيل processAllAtOnce ()

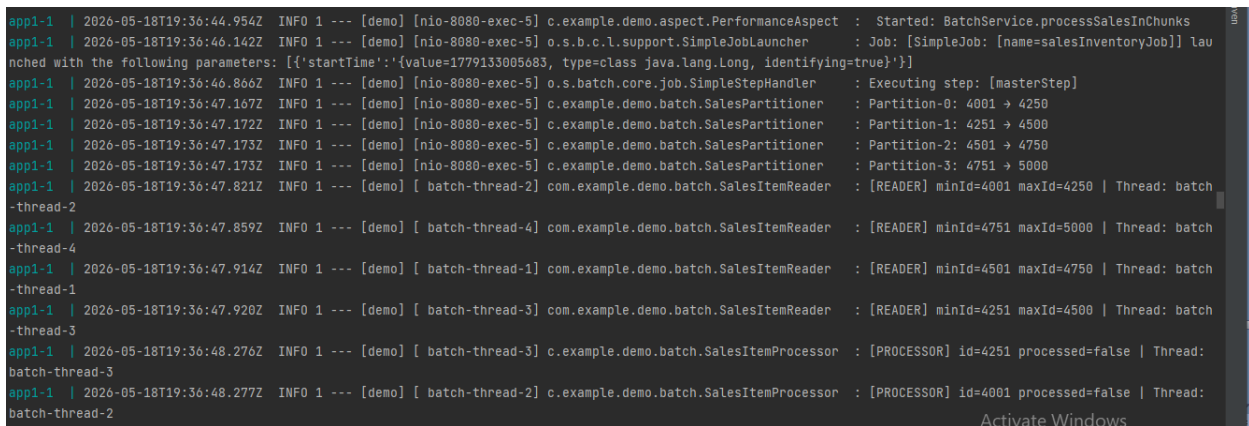
[ALL-AT-ONCE] Processed: 1000 records
 هنا النظام عالج 1000 سجل (record) دفعة واحدة.

Finished: BatchService.processAllAtOnce | Time: 27102ms
 وقت التنفيذ 27

ثانيا : بعد التحسين



استغرق الامر 11 ms وهذا الوقت على بيانات كبيرة يفرق جدا في الاداء



Executing step: [masterStep]
 هنا تم تشغيل ال masterstep وهو المسؤول عن تقسيم البيانات partition

تقسيم بيانات المبيعات إلى أجزاء (Ranges)

بدل ما يعالج كل البيانات دفعة واحدة، قسمها إلى 4 أجزاء:

- Partition 0: من 4001 إلى 4250
- Partition 1: من 4251 إلى 4500

- Partition 2: من 4501 إلى 4750
- Partition 3: من 4751 إلى 5000

batch-thread-1
batch-thread-2
batch-thread-3
batch-thread-4

هون واضح انو النظام شغل 4 thread بالتوازي كل Thread مسؤول عن Partition معينة.

وبعدين صار يطبق المراحل تبعو (Read → Process → Write)

ملخص بسيط :

تم تنفيذ Batch Processing لمعالجة بيانات المبيعات الضخمة باستخدام أسلوب Chunk Processing لتحسين الأداء. بدلاً من تحميل جميع السجلات مرة واحدة، يتم تقسيم البيانات إلى دفعات صغيرة (Chunks)، حيث تتم معالجة كل دفعة بشكل مستقل. كما تم تشغيل العملية كـ Background Job باستخدام @Scheduled، ليتم تنفيذها تلقائياً يومياً الساعة الثانية صباحاً، مما يقلل الضغط على النظام ويحسن استهلاك الذاكرة وزمن التنفيذ. عند معالجة 1000 سجل، انخفض زمن التنفيذ من حوالي 27 ثانية عند استخدام processAllAtOnce () إلى حوالي 11 ثانية باستخدام Chunks.

الطلب الخامس:

مع ازدياد عدد المستخدمين والطلبات في الأنظمة أصبح الاعتماد على Server واحد غير كافٍ لتقديم أداء مستقر وفعال، لأن ذلك يؤدي إلى مشاكل عديدة مثل:

- زيادة الضغط على الخادم
- بطء الاستجابة
- اختناق النظام عند ارتفاع عدد الطلبات
- عدم استغلال الموارد بشكل متوازن
- احتمالية توقف الخدمة بالكامل عند فشل السيرفر

الهدف من هذا الطلب لم يكن فقط توزيع الطلبات على عدة خوادم وإنما بناء آلية تراعي:

- الحمل الحالي على كل خادم
- قدرة كل خادم على المعالجة
- الأداء تحت الضغط
- التزامن بين الطلبات Concurrent Requests
- قابلية التوسع مستقبلاً

المشكلة لم تكن فقط إرسال الطلبات إلى عدة Servers ، وإنما: كيف يمكن توزيع الطلبات بشكل ذكي وعادل مع مراعاة حالة كل سيرفر وقدرته الفعلية؟ لأن السيرفرات في الواقع ليست متساوية دائماً فقد يكون:

- أحد السيرفرات أقوى من غيره
- أحدها مشغولاً حالياً بعدد كبير من الطلبات
- أحدها يمتلك موارد معالجة أكبر

وبالتالي فإن توزيع الطلبات بشكل عشوائي أو ثابت قد يؤدي إلى:

- ضغط زائد على بعض السيرفرات
- ضعف استغلال السيرفرات الأقوى
- انخفاض الأداء العام للنظام

ولتحقيق ذلك قمنا باستخدام :

- Docker لمحاكاة عدة سيرفرات مستقلة
- خوارزمية Hypaired تجمع بين :
 - Least Connections
 - Weighted Distribytion

دراسة الخوازميات الممكنة :

قبل اختيار الحل النهائي قمنا بدراسة عدة خوارزميات Load Balancing :
الحلول المقترحة :

أولاً : Random Load Balancing

في هذه الطريقة يتم اختيار السيرفر بشكل عشوائي لكل طلب

مثال:

- Request 1 → app1
- Request 2 → app3
- Request 3 → app1

لماذا لم نختار هذا الحل :

لأن :

- التوزيع غير مستقر
- لا يراعي الضغط الحالي على السيرفر
- لا يراعي قوة السيرفرات
- قد يحصل سيرفر ضعيف على عدد كبير من الطلبات

لذلك اعتبرناه غير مناسب لنظامنا.

ثانياً : Round Robin

في هذه الطريقة يتم توزيع الطلبات بالتتابع بين السيرفرات : app1 → app2 → app3 → app1
لماذا لم نستخدمه :

رغم أن هذه الطريقة تُعدّ أفضل من Random إلا أنها تفترض أن جميع السيرفرات متساوية أي جميعها نفس القدرة بينما في نظامنا :

- بعض السيرفرات أقوى من غيرها
- بعض السيرفرات تمتلك قدرة معالجة أكبر

وبالتالي توزيع نفس عدد الطلبات على جميع السيرفرات يُعتبر قرار غير واقعي

ثالثاً : Weighted Round Robin

في هذه الطريقة يحصل السيرفر الأقوى على عدد أكبر من الطلبات اعتماداً على قيمة ال weight
مثال : بما أننا وضعنا لل app2 أعلى وزن 3 لذلك يحصل على طلبات أكثر

ولكن لماذا لم يكن كافياً أيضاً ؟

لأن هذا الحل يراعي فقط قوة السيرفر لكنه لا يراعي الحمل الحالي
قد يكون السيرفر قوياً لكنه مشغول حالياً بعدد كبير من الطلبات وبالتالي قد يستمر النظام بإرسال Request إضافية إليه رغم أنه مشغول .

وهذا قد يؤدي إلى:

- زيادة الضغط على السيرفر الأقوى
- تأخير بعض الطلبات
- اختلال التوازن أثناء التشغيل الفعلي

قرارنا النهائي :

بعد دراسة الحلول السابقة قررنا استخدام : Weighted Least Connections

وهي خوارزمية Hypired تجمع بين :

التقنية	الهدف
---------	-------

التقنية	الهدف
Least Connections	معرفة أقل السيرفرات انشغالاً
Weight	مراعاة قوة السيرفر

نظامنا يتكوّن من :

المكوّن	الوظيفة
Load Balancer	استقبال الطلبات واختيار أفضل Server
Worker Servers	تنفيذ الطلبات الفعلية
Docker Containers	محاكاة السيرفرات المستقلة
Logging System	مراقبة الأداء وتتبع التنفيذ

بنية النظام (System Architecture) :

قمنا ببناء عدّة Containers حيث يحتوي النظام على :

الدور	Container
Load Balancer	lb
Worker Instance	app1
Worker Instance	app2
Worker Instance	app3
قاعدة البيانات	mysql
التخزين المؤقت Cache	redis

لماذا استخدمنا Docker في هذا الطلب بسبب توفّر عدّة ميزات منها:

- تشغيل عدّة Instances بسهولة
- عزل الخدمات عن بعضها
- إنشاء شبكة داخلية بين السيرفرات
- سهولة الاختبار والتوسع
- إمكانية إعادة تشغيل النظام بالكامل بأمر واحد

لماذا قمنا بإنشاء عدّة Worker Servers؟

في الأنظمة الحقيقية لا يتم الاعتماد على سيرفر واحد فقط، لأن ذلك يؤدي إلى:

- زيادة الضغط
- نقطة فشل وحيدة Single Point of Failure
- انخفاض الأداء عند زيادة المستخدمين

لذلك قمنا بإنشاء عدّة instance :

Server	Weight
app1	1
app2	3
app3	2

كل Server يمثل سيرفر مختلف بقدرات معالجة مختلفة.

لماذا استخدمنا نفس التطبيق لكل الـ **Instances** ؟

قمنا باستخدام نفس نسخة التطبيق لكل السيرفرات لكن قمنا بتغيير السلوك عبر Environment Variables عن طريق :

▪ APP_ROLE

▪ APP_NAME

وهذا سمح لنا بـ :

▪ تقليل تكرار الكود

▪ سهولة التوسّع

▪ تشغيل عدّة أدوار بنفس التطبيق

لماذا استخدمنا **Weight** ؟

الـ Weight يمثل قدرة السيرفر على التحمّل فمثلاً:

▪ app1 : سيرفر ضعيف

▪ app2 : سيرفر قوي

▪ app3 : سيرفر متوسط

وبالتالي لا يحصل جميع السيرفرات على نفس عدد الطلبات لأن السيرفر الأقوى يجب أن يتحمّل عدد أكبر من الـ Request .

كيف تعمل الخوارزمية :

قمنا بحساب قيمة لكل سيرفر :

$$\text{Score} = \text{activeConnections} / \text{weight}$$

ثم نقوم باختيار السيرفر ذو أقل قيمة (Score)

حيث:

▪ activeConnections : عدد الطلبات الحالية

▪ weight : قدرة السيرفر

مثال عملي بسيط قبل عرض صور الاختبار :

Server	Active	Weight	Score
app1	1	1	1.0

Server	Active	Weight	Score
app2	2	3	0.66
app3	1	2	0.5

رغم أن app2 لديه طلبات أكثر من app1 إلا أن قدرته أعلى لذلك يبقى مؤهلاً لاستقبال طلبات إضافية كما عالجت حالات تعادل ل Score (Tie Handling) :
 في بعض الحالات قد تمتلك عدة سيرفرات نفس ل Score لذلك قمنا بإضافة قاعدة إضافية وهي :
 إذا تساوت الـ Scores يتم اختيار السيرفر الأعلى Weight لأن السيرفر الأقوى أكثر قدرة على تحمل الضغط المستقبلي

لماذا اعتبرناه الحل الأفضل :

لأن الخوارزمية :

- منع الضغط الزائد على السيرفرات الضعيفة
- دعم العمل تحت الضغط العالي
- تمنع اختناق السيرفرات الضعيفة
- توزيع الأحمال بشكل متوازن
- استغلال السيرفرات القوية بشكل أفضل

: LoadBalancingService

قمنا ببناء Service مسؤولة عن :

- تحميل السيرفرات
- حساب الـ Scores
- اختيار أفضل سيرفر
- تحديث عدد الاتصالات الحالية
- تحرير السيرفر بعد انتهاء الطلب

: لماذا استخدمنا synchronized

النظام لدينا يتعامل مع عدة طلبات متزامنة وهذا قد يسبب Race Condition حيث يمكن لعدة Threads اختيار نفس السيرفر بنفس اللحظة إذا لم تتم حماية عملية الاختيار

وهكذا نكون قد ضمنا :

- سلامة البيانات
- منع التضارب
- الحفاظ على دقة عدد الاتصالات الحالية لكل Server

لماذا لم نستخدم Pessimistic Locking من الطلب الأول :

لأنه مخصص لحماية البيانات داخل قاعدة البيانات DB بينما مشكلتنا الحالية In-Memory Concurrency أي أن البيانات موجودة داخل RAM وليس داخل Database لذلك استخدمنا في هذا الطلب synchronized

لماذا لم نستخدم ReentrantLock أو AtomicInteger ؟

قمنا بدراسة حلول أخرى مثل:

- ReentrantLock
- AtomicInteger
- Concurrent Collections

لكننا لم نستخدمها لأن:

- النظام صغير نسبياً
- synchronized أبسط وأوضح
- يحقق Thread Safety بشكل كافٍ

ما الفرق الـ Load Balancer عن الـ Workers ؟

- الـ Worker مسؤول عن تنفيذ الطلبات الفعلية ومعالجة العمليات داخل كل Instance
- أما الـ Load Balancer مسؤول عن استقبال الطلبات القادمة واختيار أفضل سيرفر اعتماداً على خوارزمية :

Weighted Least Connections

حيث:

- **LoadBalancerService** مسؤول عن :
 - حساب الحمل الحالي على السيرفرات
 - تطبيق خوارزمية الاختيار
 - توجيه الطلب إلى أفضل Instance
- **Worker Instances** مسؤولة عن:

- تنفيذ الطلبات الفعلية
 - معالجة العمليات
 - التعامل مع قاعدة البيانات
 - إرجاع النتائج للمستخدم
- وبالتالي حافظ النظام على الفصل الوظيفي بين:

- منطق توزيع الحمل
- ومنطق تنفيذ العمليات

وأيضاً لقد استعنا بـ فايل **logger** الذي استخدمناه لتحقيق مبدأ لـ **AOP** من أجل أن يسجل النظام :

- السيرفر المختار
- عدد الاتصالات الحالية
- الـ Score لكل سيرفر
- زمن التنفيذ
- اسم الثريد
- تحرير السيرفر بعد انتهاء الطلب

النتيجة النهائية التي حصلنا عليها :
أصبح نظامنا يتميز ب:

- توزيع أحمال ذكي
- دعم الـ Concurrency
- مراعاة قوة السيرفرات
- منع Race Conditions
- قابلية التوسع
- مراقبة الأداء
- محاكاة بيئة Distributed حقيقية

كما أن الحل الذي اعتمدناه يتفوق على:

- Random
- Round Robin
- Weighted Round Robin

لأنه يعتمد على:

- الحمل الحقيقي الحالي
- قوة السيرفر
- التوازن الديناميكي أثناء التشغيل

ولذلك اعتبرنا أن: Weighted Least Connections

هو القرار الهندسي الأنسب لتحقيق متطلبات مشروعنا

الاختبار : قبل وبعد التحسين :

قبل التحسين: الضغط كامل على السيرفر الاول

app1-1		2026-05-18T19:50:08.534Z	INFO	1	---	[demo]	[omcat-handler-3]	c.e.d.c.WithoutLoadBalancerController	: Request handled ONLY by app1
app1-1		2026-05-18T19:50:08.585Z	INFO	1	---	[demo]	[omcat-handler-4]	c.e.d.c.WithoutLoadBalancerController	: Request handled ONLY by app1
app1-1		2026-05-18T19:50:08.634Z	INFO	1	---	[demo]	[omcat-handler-5]	c.e.d.c.WithoutLoadBalancerController	: Request handled ONLY by app1
app1-1		2026-05-18T19:50:08.686Z	INFO	1	---	[demo]	[omcat-handler-6]	c.e.d.c.WithoutLoadBalancerController	: Request handled ONLY by app1
app1-1		2026-05-18T19:50:08.733Z	INFO	1	---	[demo]	[omcat-handler-7]	c.e.d.c.WithoutLoadBalancerController	: Request handled ONLY by app1
app1-1		2026-05-18T19:50:08.786Z	INFO	1	---	[demo]	[omcat-handler-8]	c.e.d.c.WithoutLoadBalancerController	: Request handled ONLY by app1
app1-1		2026-05-18T19:50:08.835Z	INFO	1	---	[demo]	[omcat-handler-9]	c.e.d.c.WithoutLoadBalancerController	: Request handled ONLY by app1
app1-1		2026-05-18T19:50:08.887Z	INFO	1	---	[demo]	[mcat-handler-10]	c.e.d.c.WithoutLoadBalancerController	: Request handled ONLY by app1
app1-1		2026-05-18T19:50:08.935Z	INFO	1	---	[demo]	[mcat-handler-11]	c.e.d.c.WithoutLoadBalancerController	: Request handled ONLY by app1
app1-1		2026-05-18T19:50:08.984Z	INFO	1	---	[demo]	[mcat-handler-12]	c.e.d.c.WithoutLoadBalancerController	: Request handled ONLY by app1
app1-1		2026-05-18T19:50:09.035Z	INFO	1	---	[demo]	[mcat-handler-13]	c.e.d.c.WithoutLoadBalancerController	: Request handled ONLY by app1
app1-1		2026-05-18T19:50:09.085Z	INFO	1	---	[demo]	[mcat-handler-14]	c.e.d.c.WithoutLoadBalancerController	: Request handled ONLY by app1
app1-1		2026-05-18T19:50:09.134Z	INFO	1	---	[demo]	[mcat-handler-15]	c.e.d.c.WithoutLoadBalancerController	: Request handled ONLY by app1
app1-1		2026-05-18T19:50:09.184Z	INFO	1	---	[demo]	[mcat-handler-16]	c.e.d.c.WithoutLoadBalancerController	: Request handled ONLY by app1
app1-1		2026-05-18T19:50:09.234Z	INFO	1	---	[demo]	[mcat-handler-17]	c.e.d.c.WithoutLoadBalancerController	: Request handled ONLY by app1
app1-1		2026-05-18T19:50:09.287Z	INFO	1	---	[demo]	[mcat-handler-18]	c.e.d.c.WithoutLoadBalancerController	: Request handled ONLY by app1
app1-1		2026-05-18T19:50:09.334Z	INFO	1	---	[demo]	[mcat-handler-19]	c.e.d.c.WithoutLoadBalancerController	: Request handled ONLY by app1

without_load.jmx (C:\intelli\ecommerce_backend\src\main\java\com\example\demo\jmeter_files\without_load.jmx) - Apache JMeter (5.6.3)

00:00:02 0 0/20

Test Plan
Thread Group
HTTP Request
View Results Tree
View Results in Table
Aggregate Report

View Results in Table

Name: View Results in Table

Comments:

Write results to file / Read from file

Filename: Log/Display Only: ☐ Errors ☐ Successes

Sample #	Start Time	Thread Name	Label	Sample Time(ms)	Status	Bytes	Sent Bytes	Latency	Connect Time(ms)
1	22:40:52.910	Thread Group 1-2	HTTP Request	2060	✓	184	141	2060	1
2	22:40:52.861	Thread Group 1-1	HTTP Request	2109	✓	184	141	2109	1
3	22:40:52.960	Thread Group 1-3	HTTP Request	2010	✓	184	141	2010	1
4	22:40:53.009	Thread Group 1-4	HTTP Request	2007	✓	184	141	2007	1
5	22:40:53.062	Thread Group 1-5	HTTP Request	2008	✓	184	141	2008	1
6	22:40:53.110	Thread Group 1-6	HTTP Request	2006	✓	184	141	2006	1
7	22:40:53.160	Thread Group 1-7	HTTP Request	2006	✓	184	141	2006	0
8	22:40:53.211	Thread Group 1-8	HTTP Request	2005	✓	184	141	2005	0
9	22:40:53.260	Thread Group 1-9	HTTP Request	2006	✓	184	141	2006	0
10	22:40:53.311	Thread Group 1-10	HTTP Request	2006	✓	184	141	2006	0
11	22:40:53.362	Thread Group 1-11	HTTP Request	2005	✓	184	141	2005	0
12	22:40:53.410	Thread Group 1-12	HTTP Request	2006	✓	184	141	2006	0
13	22:40:53.460	Thread Group 1-13	HTTP Request	2005	✓	184	141	2005	1
14	22:40:53.509	Thread Group 1-14	HTTP Request	2007	✓	184	141	2007	1
15	22:40:53.560	Thread Group 1-15	HTTP Request	2007	✓	184	141	2007	1
16	22:40:53.610	Thread Group 1-16	HTTP Request	2005	✓	184	141	2005	1
17	22:40:53.661	Thread Group 1-17	HTTP Request	2008	✓	184	141	2008	2
18	22:40:53.710	Thread Group 1-18	HTTP Request	2006	✓	184	141	2006	1
19	22:40:53.761	Thread Group 1-19	HTTP Request	2007	✓	184	141	2007	1
20	22:40:53.822	Thread Group 1-20	HTTP Request	2007	✓	184	141	2007	1

☐ Scroll automatically? ☐ Child samples? No of Samples: 20 Latent Sample: 2007 Average: 4014 Deviation: 24

without_load.jmx (C:\intelli\ecommerce_backend\src\main\java\com\example\demo\jmeter_files\without_load.jmx) - Apache JMeter (5.6.3)

00:00:02 0 0/20

Test Plan
Thread Group
HTTP Request
View Results Tree
View Results in Table
Aggregate Report

View Results Tree

Name: View Results Tree

Comments:

Write results to file / Read from file

Filename: Log/Display Only: ☐ Errors ☐ Successes

Search: ☐ Case sensitive ☐ Regular exp.

Text

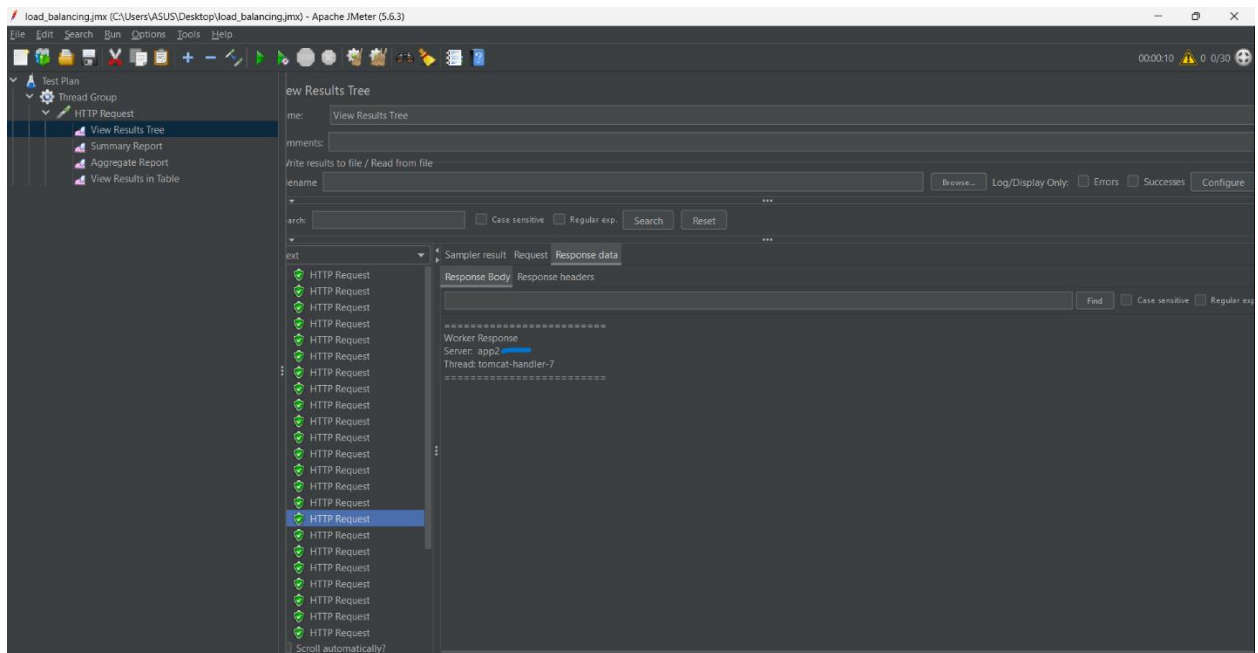
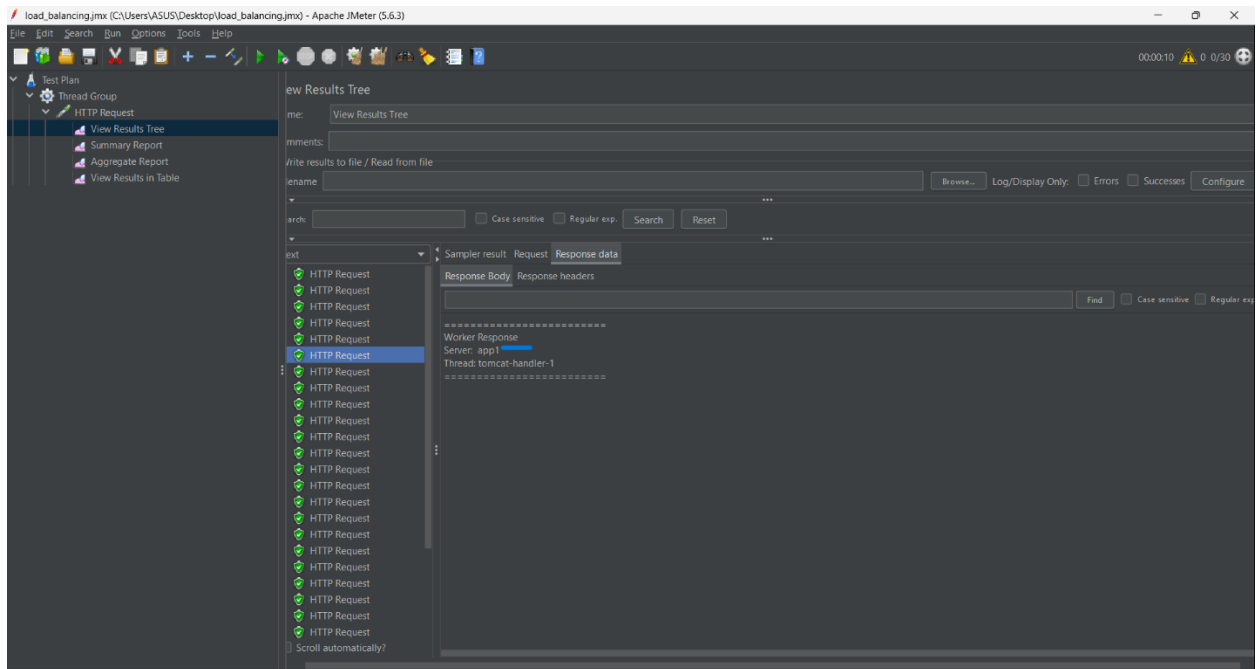
Sampler result Request Response data

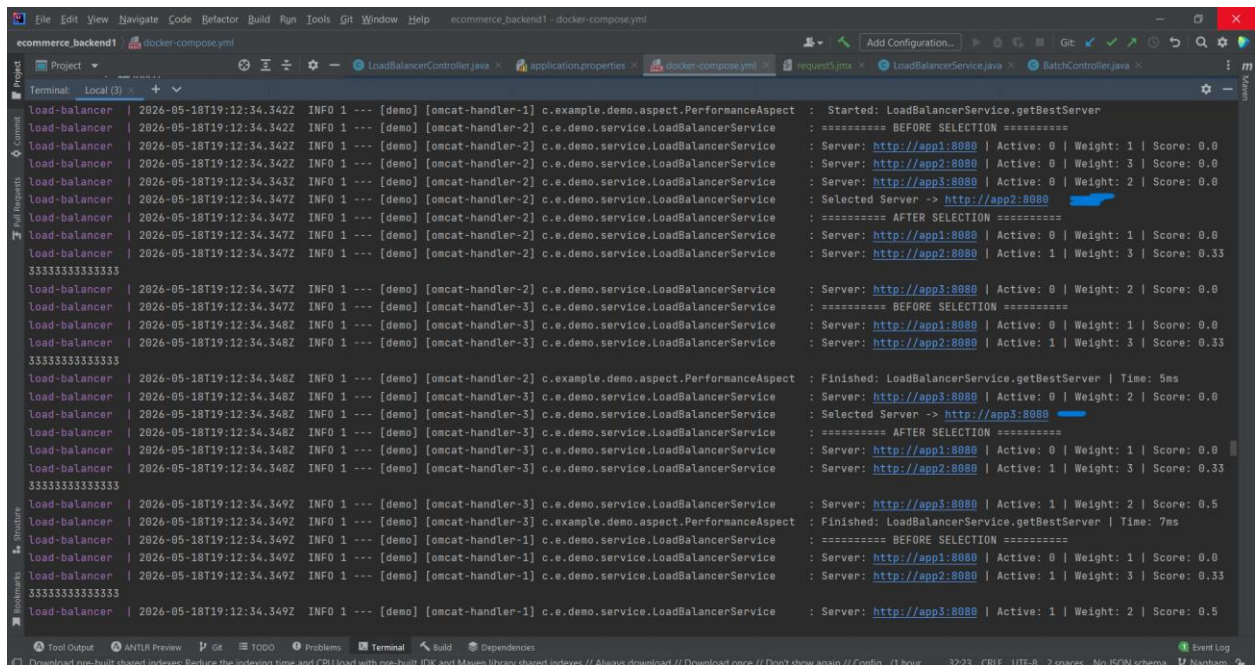
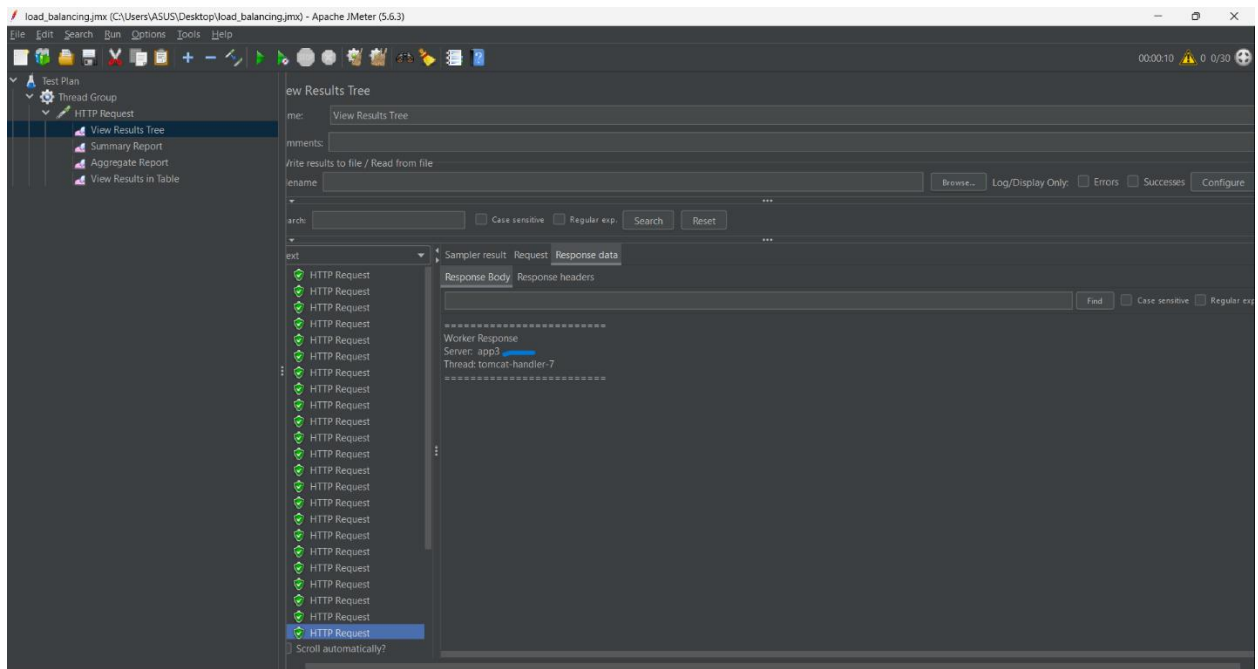
Response Body Response headers

Handled ONLY by: app1

☐ Scroll automatically?

بعد التحسين : تم توزيع الطلبات كما تحدثنا حيث تم وضع اشارات على الصور





load_balancing.jmx (C:\Users\ASUS\Desktop\load_balancing.jmx) - Apache JMeter (5.6.3)

File Edit Search Run Options Tools Help

Test Plan

- Thread Group
 - HTTP Request
 - View Results Tree
 - Summary Report
 - Aggregate Report
 - View Results in Table

View Results in Table

Name: View Results in Table

Comments:

Write results to file / Read from file

Filename: Browse... Log/Display Only: ☐ Errors ☐ Successes ☐ Configure

Sample #	Start Time	Thread Name	Label	Sample Time(ms)	Status	Bytes	Sent Bytes	Latency	Connect Time(ms)
1	22:12:34.328	Thread Group 1-4	HTTP Request	10307	✓	270	132	10306	0
2	22:12:34.460	Thread Group 1-8	HTTP Request	10175	✓	270	132	10174	1
3	22:12:34.561	Thread Group 1-11	HTTP Request	10075	✓	270	132	10075	1
4	22:12:34.263	Thread Group 1-2	HTTP Request	10373	✓	270	132	10373	0
5	22:12:34.361	Thread Group 1-5	HTTP Request	10276	✓	270	132	10276	1
6	22:12:34.494	Thread Group 1-9	HTTP Request	10144	✓	270	132	10144	1
7	22:12:34.296	Thread Group 1-3	HTTP Request	10343	✓	270	132	10343	1
8	22:12:34.527	Thread Group 1-10	HTTP Request	10112	✓	270	132	10112	1
9	22:12:34.593	Thread Group 1-12	HTTP Request	10046	✓	270	132	10046	0
10	22:12:34.394	Thread Group 1-6	HTTP Request	10248	✓	270	132	10247	2
11	22:12:34.229	Thread Group 1-1	HTTP Request	10418	✓	270	132	10418	1
12	22:12:34.428	Thread Group 1-7	HTTP Request	10221	✓	270	132	10221	1
13	22:12:34.625	Thread Group 1-13	HTTP Request	10026	✓	270	132	10026	1
14	22:12:34.658	Thread Group 1-14	HTTP Request	10014	✓	270	132	10014	0
15	22:12:34.690	Thread Group 1-15	HTTP Request	10012	✓	270	132	10012	1
16	22:12:34.724	Thread Group 1-16	HTTP Request	10022	✓	270	132	10022	1
17	22:12:34.757	Thread Group 1-17	HTTP Request	10011	✓	270	132	10011	1
18	22:12:34.789	Thread Group 1-18	HTTP Request	10012	✓	270	132	10012	1
19	22:12:34.823	Thread Group 1-19	HTTP Request	10011	✓	270	132	10011	0
20	22:12:34.856	Thread Group 1-20	HTTP Request	10012	✓	270	132	10012	1
21	22:12:34.889	Thread Group 1-21	HTTP Request	10012	✓	270	132	10011	0
22	22:12:34.922	Thread Group 1-22	HTTP Request	10012	✓	271	132	10012	0
23	22:12:34.954	Thread Group 1-23	HTTP Request	10011	✓	270	132	10011	1
24	22:12:34.988	Thread Group 1-24	HTTP Request	10011	✓	271	132	10010	1
25	22:12:35.021	Thread Group 1-25	HTTP Request	10011	✓	271	132	10011	0

☐ Scroll automatically? ☐ Child samples? No of Samples: 30 Latest Sample: 10011 Average: 10000 Deviation: 1.00

الطلب السادس:

في نظام التجارة الالكترونية عادة يكون هناك عمليات كثيرة جداً من نوع قراءة على المنتجات، مثل:

- GET /products على المنتجات.
- GET /products/{id} على منتج معين في قسم ما.
- GET /products/top-selling على المنتجات الأعلى مبيعاً
- GET /categories على الفئات.
- GET /reports/daily-sales على المبيعات اليومية.

هذه العمليات قد يتكرر عرضها الاف المرات. وبدون مكان تخزين سريع وقريب، سيتوجب على كل طلب من المستخدم ان ينشئ اتصال مع قاعدة البيانات، وبالتالي إذا أصبح لدينا 100 مستخدم متزامن في نفس الوقت، وربما لاحقاً 1000 ستظهر مشاكل لدينا:

- كثرة DB queries
- ارتفاع response time
- ضغط على connection pool
- زيادة lock/contention أحياناً
- بطء صفحات المنتجات
- انخفاض throughput

هنا تأتي فكرة الـ **cache**، حيث تكون ذاكرة صغيرة وسريعة جداً تمكننا من حفظ البيانات لفترة مؤقتة، بحيث تكون مناسبة عندما نحتاج خدمة قراءات متكررة بزمان منخفض جداً وتقليل الحمل على قاعدة

البيانات الأساسية، مع قبول فكرة أن البيانات في الكاش قد تكون قديمة ضمن حدود (TTL Time To Live).

وبما أننا استخدمنا في الطلب السابق Load balancer، الذي أتاح لنا عملية قوية لتوزيع الطلبات على نسخ متعددة من التطبيق، سنلجأ إلى استخدام Distributed Cache ليكون طبقة تخزين مشتركة بين أكثر من instance.

ومن أفضل أنواع الـ Distributed Cache المتوفرة لدينا، هي الـ Redis. ليكون لدينا نوعين من التخزين:

- Database = Source of Truth مصدر الحقيقة أو مصدر البيانات الأساسي.
- Redis نسخة مؤقتة لتحسين الأداء.

ولكن لم نصل إلى قرار اختيار الـ Redis، إلا بعد دراسة عدة حلول وبدائل، منها:

1. No Cache (الحل الافتراضي قبل تطبيق الـ cache):

لم نختره كحل نهائي، لكنه ليس دائماً خطأ. يكون مناسباً عندما يكون النظام صغيراً، وعدد المستخدمين قليل، وقراءات قاعدة البيانات ليست مكلفة. مثلاً تطبيق إداري داخلي يستخدمه 5 موظفين فقط، أو مشروع في بدايته حيث تكلفة إضافة Redis أعلى من فائدته.

لكنه **غير مناسب** عندما لأن مشروع التجارة الإلكترونية فيه قراءات متكررة جداً للمنتجات والتقارير، ومع وجود مئات والاف المستخدمين المتزامنين في نفس الوقت، قاعدة البيانات ستتعرض لضغط غير ضروري.

2. Local Cache داخل كل Instance

مثلاً نستخدم ConcurrentHashMap أو Caffeine داخل تطبيق Spring. هذا مناسب جداً في أنظمة تعمل على instance واحدة، أو عندما تكون البيانات صغيرة وثابتة تقريباً مثل:

- قائمة الدول
- قائمة العملات
- إعدادات ثابتة
- ثوابت UI

ومميزته أن هذا الحل سريع جداً لأنه داخل الذاكرة ولا يحتاج اتصال شبكة. **ولكنه غير مناسب** كحل رئيسي عندما لأن الطلب الخامس جعل النظام يعمل خلف Load Balancer بعدة instances فلا نريد أن يكون هناك local cache لكل instance، لأن هذا سيسبب:

- تكرار غير ضروري
- اختلاف في البيانات بين instances
- صعوبة invalidation للبيانات المحفوظة داخل كل cache.
- ضعف consistency بين كل نسخة وأخرى.

3. Redis Distributed Cache (الحل الذي اخترناه)

يكون مناسباً عندما يكون عندنا أكثر من application instance وتحتاج كلها إلى طبقة cache مشتركة. مثل:

- Product details
- Top-selling products
- Daily sales reports
- Categories
- Homepage data

- **ميزته** أنه يجعل الكاش مشتركاً Redis → App1 / App2 / App3

- لكن Redis ليس مناسباً كمصدر حقيقة للبيانات الحرجة مثل:

- stock decrement
- payment status
- final order state

وبالتالي عند استخدام redis سنمنع تخزين البيانات الحرجة، التي تتطلب ان تكون دقيقة وجديدة، التي تتطلب ان لا تحوي أي نوع من staleness.

4. CDN Cache

CDN مناسب جداً للملفات الثابتة أو شبه الثابتة:

- صور المنتجات
- CSS
- JavaScript
- static assets
- ملفات PDF عامة

وهو ممتاز إذا كان لدينا مستخدمون من مناطق مختلفة جغرافياً. لكنه لا يحل مباشرة المشكلة التي لدينا:

- GET /products/{id} جلب تفاصيل ومعلومات منتج معين.
- DB queries
- عرض عدد المنتجات وتفاصيلها في الصفحة الرئيسية Home Page التي يرتفع بها عدد الزيارات.

لأن هذه APIs ديناميكية وتحتاج TTL و invalidation من التطبيق. وبالتالي:

- CDN مناسب للصور والملفات الثابتة.
- Redis مناسب أكثر للـ API data داخل backend التي يتكرر استعلامها في قاعدة البيانات بدون ان تتغير كثيراً.

رغم ذلك: اختيار Redis لا يعني ان الأمور انتهت عند هذا الحد، حيث يكون التحدي الحقيقي كيف نضيف طبقة Cache وما نمط الكتابة فيها، وماذا نخزن في Redis وماذا لا نخزن، وكيف سنضبط الـ TTL مع كيفية عمل Invalidation للبيانات التي تغيرت، التي تكون من أصعب التحديات لدينا.

Cache Patterns الأساسية:

1. Cache-Aside / Lazy Loading

النمط الذي سنختاره لمشروعنا، وتقوم فكرته على أن:

1. التطبيق يسأل Redis.
2. إذا وجد البيانات، يرجعها مباشرة.
3. إذا لم يجدها، يقرأ من DB.
4. يخزن النتيجة في Redis مع TTL.
5. يرجع النتيجة للمستخدم.

لماذا يكون هذا النمط مناسب لنا؟ ذلك لأنه:

- بسيط
- واضح
- مناسب للقراءات الكثيرة
- لا يخزن إلا البيانات المطلوبة فعلاً
- سهل القياس قبل/بعد

2. Write-Through

تقوم فكرته على أنه كلما كتبنا في قاعدة البيانات نكتب أيضاً في الـ **cache**.
مثلاً عند تحديث المنتج:

- Update product in DB
- Update product in Redis
-

الميزة:

- الكاش أحدث غالباً

العيوب:

- كتابة أبطأ
- تعقيد أعلى
- مشكلة لو DB نجحت و Cache فشل أو العكس
- قد نحتاج transaction/outbox

يكون مناسباً عندما نريد أن يبقى الكاش قريباً جداً من آخر تحديث، وعندما تكون الكتابات قليلة نسبياً والقراءات كثيرة. مثلاً:

- إعدادات النظام
- ملف مستخدم نادر التعديل
- قائمة أسعار يتم تحديثها من لوحة Admin
- بيانات حساسة مثل رصيد بنكي.

ولكنه **ليس خيارنا الأول** وذلك لأنه يزيد التعقيد ويفتح مشكلة أن القاعدة والكاش يجب ان ينجحا مع بعضهما. كما ان ال Use Cases التي لدينا لا تتطلب مثل هكذا أنماط من الكتابة.

3. Write-Behind / Write-Back

هذا النمط يعني أن نكتب أولاً في الكاش، ثم لاحقاً تتم الكتابة في قاعدة البيانات. يكون مناسباً في أنظمة تعمل eventual persistence ، مثل:

- عدادات analytics
 - عدد مشاهدات منتج
 - logs مؤقته
 - click tracking
 - metrics
 - إعجابات وسائل التواصل الاجتماعي.
- مثلاً: عدد مشاهدات المنتج يمكن تحديثه في Redis بسرعة ثم تفريغه لاحقاً إلى DB.
 - لكنه **خطير جداً** للمخزون والدفع والطلبات، لأن فقدان Redis أو فشل التفريغ يعني فقدان بيانات مالية أو تجارية حساسة.

إذن **Write-behind** مناسب للبيانات التحليلية أو غير الحرجة. ومرفوض لحالتنا للمخزون والدفع وإنشاء الطلب.

Cache Invalidation:

هذه أصعب نقطة في الكاش بحيث إزالة البيانات من الكاش عندما تصبح غير صالحة أو غير مفيدة. بحيث تكون المشكلة التي لدينا:

- البيانات تغيرت في DB لكن Redis ما زال يحمل النسخة القديمة

هنا يتوجب علينا أن نقرر كيف سنلغي الكاش؟

يوجد عدة استراتيجيات Invalidation قمنا بدراستها والمقارنة فيما بينها:

1. TTL فقط.

نترك البيانات تنتهي بعد وقت معين.

الميزة:

- بسيط

العيب:

- قد يرى المستخدم بيانات قديمة حتى نهاية TTL

2. Evict on Write

عند تحديث المنتج في DB ، نحذف المفتاح من Redis

• Update product in DB

• Delete product:10 from Redis

ثم أول قراءة لاحقة تعيد تحميله من DB.

القرار الذي سنأخذه في مشروعنا سيكون حسب **use cases** الموجودة لدينا وهو عند تحديث المنتج سنقوم بـ:

1. تحديث DB

2. ثم نعمل evict لـ product:{id} ، ثم عندما أول مستخدم يستعلم عن هذا المنتج سيتم

تخزين معلومات المنتج الجديدة في الكاش.

3. Event-Based Invalidation

عند حدوث حدث، على سبيل المثال:

• ProductUpdatedEvent

• ProductPriceChangedEvent

• BatchReportGeneratedEvent

نرسل event أو ننفذ handler يحذف الكاش المتأثر.
مثلاً:

i. Batch job finishes

ii. ثم DailySalesReportGeneratedEvent

iii. ثم Evict report:daily-sales:date

هذه الاستراتيجية **قوية** ويمكن تطبيقها في حال أردنا تخزين نتائج الجرد التي قمنا بها للـ orders في الطلب الرابع في الكاش، حيث يوجد events يتم إطلاقها عند الانتهاء من batch processing التي يقوم بها الـ spring batch، بحيث نقوم بـ evict للنتيجة القديمة التي تمت معالجتها وتخزينها في الكاش لليوم الفائت.

Cache Stampede

هذه مشكلة متقدمة قليلاً حيث سنقوم بالتوسع بها في الطلب السابع ولكن سنطبقها هنا بشكل مبسط من أجل التنفيذ. **سنختار:**

- المنتج ذو الرقم 10 انتهى TTL، وفي نفس اللحظة 1000 مستخدم طلبوا المنتج، النتيجة ستكون:

• كلهم يرون cache miss

• كلهم يذهبون إلى DB

• DB يتعرض لضربة مفاجئة

هذا يسمى **Cache Stampede** أو **Thundering Herd**.

حلول المشكلة التي درسناها والحل الذي اخترناه:

1. TTL مع Random Jitter

بدل كل المنتجات تنتهي بعد 10 دقائق بالضبط:

حتى لا تنتهي مفاتيح كثيرة معاً.

2. Single-Flight / Request Coalescing

أول request يحصل على lock قصير لإعادة تحميل البيانات، والبقية تنتظر أو تحصل على stale value.

3. Serve Stale While Revalidate

إذا انتهت القيمة، يمكن إرجاع نسخة قديمة قصيرة جداً بينما تحديثها يحدث في الخلفية.
هذا متقدم، وليس ضرورياً لحالات الاستخدام التي سنطبقها.

القرار النهائي: سنستخدم الحل الأول والثاني بحيث عند وضع الـ ttl لمفتاح الـ cache الذي نريد تخزينه، سيتم إضافة عدد ثواني عشوائي يتراوح ما بين 30+/- حتى لا تنتهي كل المفاتيح في نفس اللحظة، وبفس الوقت المستخدم الذي يدخل أولاً سيضع قفلاً على قراءة المنتج، بحيث المستخدمين المتزامنين الذين يأتون في نفس الوقت، يتحم عليهم الانتظار ريثما يقوم المستخدم الأول بقراءة المنتج وتخزينه في redis، ثم بعد نجاح العملية يتم حذف هذا القفل، ليقوم بعدها المستخدمين المنتظرين بقراءة المنتجات التي تم رفعها حديثاً للـ redis.

- **الـ Use Case** التي سنطبقها في هذا الطلب ستكون ارجاع المنتجات الـ Top Selling التي يتم عرضها في سلايدر الـ Home Page، حيث يكون الاستعلام عليها كثيراً من قبل الاف مستخدمين بسبب وجودها في الصفحة الرئيسية، وبفس الوقت تفاصيل هذه المنتجات لا تتغير بشكل سريع، بالتالي عدم تخزين مثل هكذا حالة في cache سيزيد الضغط بشكل كبير على قاعدة البيانات، مما يؤدي إلى إبطاء سرعة الاستجابة، ما ينتج عنها تجربة مستخدم سيئة تضر بسمعة الشركة.

- ما سنقوم به في الكود:

- تزويد كل instance بإعداد redis في ملف Docker Compose، وذلك لكي نضمن ان نستخدم كل نسخة الكاش نفسه.
- ضبط الـ RedisCacheManager في ملف RedisCacheConfig وذلك لإمكانية تحديد خصائص الضبط الافتراضي مع إعطاء ضبط و ttl مختلف لكل cache باستخدام withInitialCacheConfiguratio.
- DTOs للكاش وليس Entities: حيث لا نخزن JPA Entity في redis، بل نخزن DTO، وذلك لأن:

1. **Entity** فيها علاقات Lazy قد تسبب مشاكل serialization .

2. **Entity** مرتبطة ب-Hibernate persistence context

3. **DTO** أكثر استقراراً ك-cache contract

4. لا نريد تخزين تفاصيل داخلية غير لازمة.

- إضافة تابع `getTopProducts` لإثبات `cache hit`، وتابع `updateProduct` الذي يقوم بتعديل تفاصيل منتج معين، وذلك لإثبات `cache invalidation`.
- كتابة **manual caching** باستخدام `RedisTemplate` بدلاً من استخدام

ال-`@cacheable annotation` الجاهزة في **spring** على تابع `getTopProducts`:

ال-`@cacheable` ممتازة للحالات البسيطة ولكنها لا تكفي دائماً، حيث نحتاج `manual caching`، عندما:

- نحتاج `TTL` ديناميكي لكل قيمة.
- نحتاج `lock` لمنع `cache stampede`.
- نحتاج `stale-while-revalidate`.
- نحتاج `cache key registry`.
- نحتاج إحصائيات دقيقة `hit/miss`.
- نحتاج التعامل مع `Redis failure` بشكل خاص.
- التعامل مع **Redis Failure**، حيث لا نريد أن ينهار النظام إذا `redis` فشل، إذ نكمل القراءة من ال-**DB**، حتى لو **أبطأ** ذلك الأداء والاستجابة.

سنبدأ الآن باختبار ال-**metrics** لدينا وذلك في حالتي تعطيل الكاش (أي قبل إضافة ال-`redis`) وبعد تفعيل الكاش:

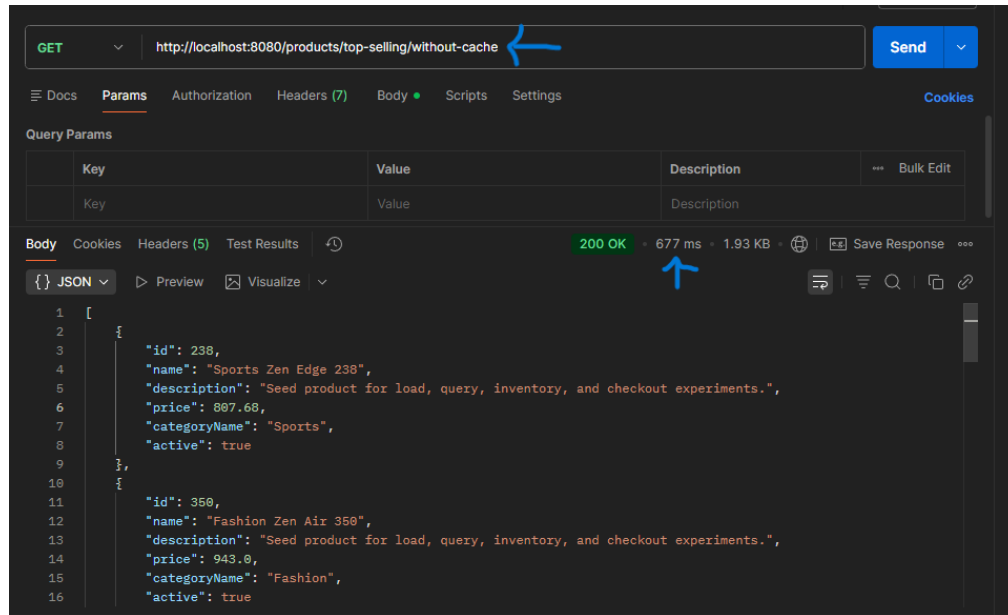
سنستخدم أداة جديدة هنا غير ال-**jmeter** التي استخدمناها في الطلبات السابقة، وهي ال-**k6** وذلك لإرسال **HTTP requests** مثل المستخدمين الحقيقيين. حيث أن **k6** يدعم سيناريوهات لتشكيل **traffic patterns** مختلفة مثل عدد **Vus (virtual users)** وجدولة **iterations**، وهذا مناسب جداً لمحاكاة ضغط واقعي على `endpoint`.

سنعتمد 100 مستخدم افتراضي، ولكن بدلاً من 100 مستخدم فجأة وفي لحظة واحدة، سنقوم بعمل **ramp-up** في ملف **k6**، حيث يكون أفضل من الضغط المفاجئ، لأنه يحاكي زيادة تدريجية في المستخدمين، حيث يزداد عدد الطلبات المرسل في اللحظة الواحدة من 0 إلى 50 ثم 100 لتبقى مدة دقيقة كاملة ثم بعد ذلك تنقص تدريجياً إلى 0. المقاييس التي سنعتمدها:

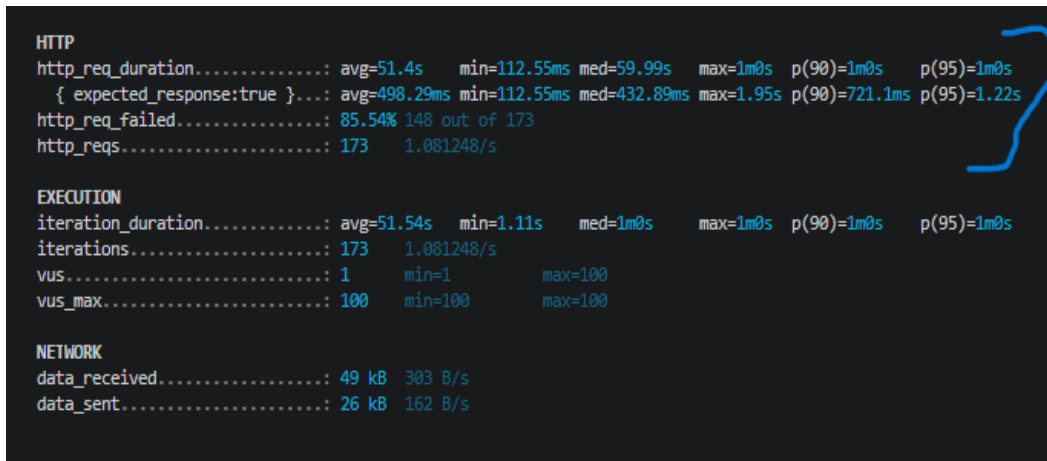
- avg response time
- p95

- p99
- throughput
- DB query count
- cache hit ratio
- Redis latency

قبل وضع الكاش:



استغرق الطلب الواحد **677ms**، يبدو رقماً جيداً ولكن عند الضغط العالي والطلبات الكثيرة تبدأ الكوارث.

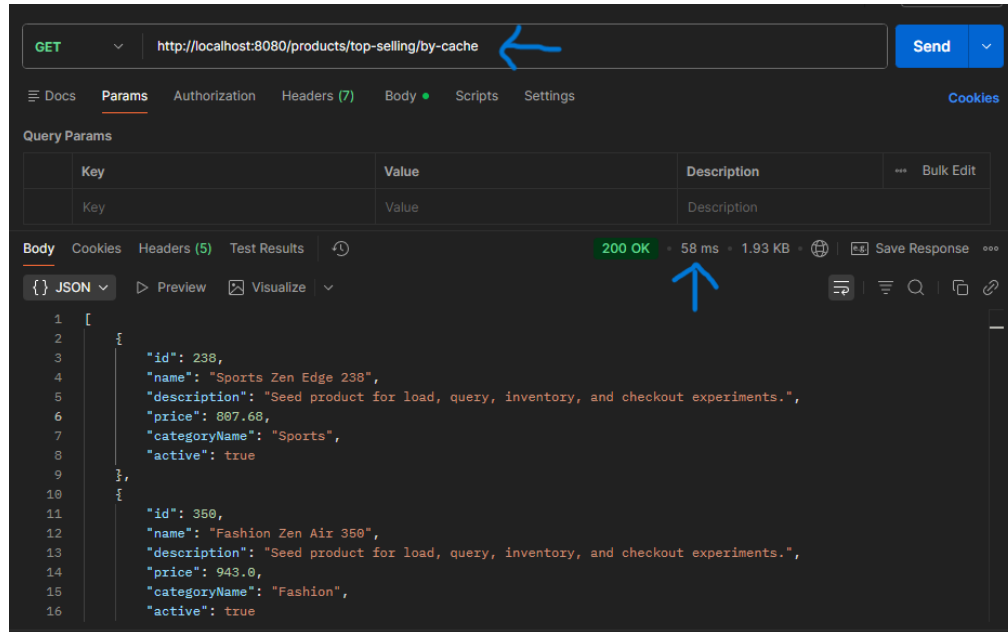


هنا الكارثة الحقيقة مع الأرقام الظاهرة بعد اختبار **k6**:

- حيث وصل متوسط سرعة كل طلب (avg) الى **51s**.
- **P(90)·P(95)=1m**: أي ان 90-95 بالمئة من الطلبات أخذت أقل او تساوي دقيقة وبالتالي 5-10 بالمئة من الطلبات أخذت فوق الدقيقة ليكتمل، وهذا رقم كارثي.

- `http_req_failed=85%`: تشير الى أن 85 بالمئة من الطلبات الاجمالية البالغة 173 (`http_reqs`)، قد فشل وأصبح `timeout`.
- `Throughput=1/s`: يدل الرقم الفاتح بجانب `http_reqs` الى ان معدل معالجة الطلبات في الثانية الواحدة هو طلب واحد.
- ما تدل عليه القيم، هو ان قاعدة البيانات قد انهارت بسبب كثرة الاتصالات في `connection_pool`، كما أدى عدد الطلبات المتزامنة الى امتلاء طبقة الـ `http`، وبالتالي الكثير من الطلبات لم تنتهي وبقيت عالقة في النظام.

بعد وضع الكاش:



- بعد وضع الكاش استغرق الطلب الواحد **58m**، أي أسرع بعشرة أضعاف من قبل.

```
TOTAL RESULTS

checks_total.....: 10936 83.54753/s
checks_succeeded...: 100.00% 10936 out of 10936
checks_failed.....: 0.00% 0 out of 10936

✓ status is 200
✓ has body

HTTP
http_req_duration.....: avg=665.79ms min=2.83ms med=50.07ms max=8.56s p(90)=2.38s p(95)=4.1s
{ expected_response:true }...: avg=665.79ms min=2.83ms med=50.07ms max=8.56s p(90)=2.38s p(95)=4.1s
http_req_failed.....: 0.00% 0 out of 5468
http_reqs.....: 5468 41.773765/s

EXECUTION
iteration_duration.....: avg=1.66s min=1s med=1.05s max=9.56s p(90)=3.38s p(95)=5.1s
iterations.....: 5468 41.773765/s
vus.....: 4 min=1 max=100
vus_max.....: 100 min=100 max=100

NETWORK
data_received.....: 11 MB 81 kB/s
data_sent.....: 591 kB 4.5 kB/s

running (2m10.9s), 000/100 VUs, 5468 complete and 0 interrupted iterations
default ✓ [=====] 000/100 VUs 2m10s
PS E:\Spring Boot\Codes\High-performance-E-Commerce> |
```

- (avg): متوسط سرعة الطلب الواحد هو 665ms، أي أفضل بعشرة أضعاف من ذي قبل.
- P(90)=2.38s، P(95)=4.1s: هنا 90-95 بالمئة من الطلبات اخذت اقل من 2 ثانية، مع انه يبدو سيئاً ان يكون هناك 5-10 بالمئة من الطلبات تستغرق 3-4 ثواني، لكن تم تفسير ذلك بسبب وجود قفل ال cache stampede، الذي يقفل الطلبات ويجعلها تنتظر لمدة نصف ثانية ريثما يقوم الطلب الحالي بإعادة التخزين في الكاش، كما أن وجود عمليات ال logs الكثيرة في تابع getTopProducts تبطئ الطلبات نوعاً ما.
- Http_req_failed=0.0: تدل قيمة الصفر على نجاح جميع الطلبات ووصولها الى الكاش بنجاح.
- http_reqs=5468: أي عدد الطلبات التي تمت معالجتها بالكامل، وهو عدد كبير ضمن مدة لا تتجاوز الدقيقتين.
- Throughput=41/s: معدل عدد الطلبات التي تم معالجته في الثانية الواحدة، هو 41 طلب.

GET <http://localhost:8080/actuator/metrics/ecommerce.db.query> Send

Query Params

Key	Value	Description
Key	Value	Description

Body: 200 OK • 2.05 s • 386 B

```
1 {
2   "name": "ecommerce.db.query",
3   "measurements": [
4     {
5       "statistic": "COUNT",
6       "value": 5.0
7     }
8   ],
9   "availableTags": [
10    {
11      "tag": "query",
12      "values": [
13        "findTopSellingProducts"
14      ]
15    }
16  ]
17 }
```

يدل هذا **response** الأعلى، على عدد الاتصالات التي تمت بقاعدة البيانات، حيث تساوي 5 فقط بعد إضافة الكاش.

GET <http://localhost:8080/actuator/metrics/ecommerce.cache.hit> Send

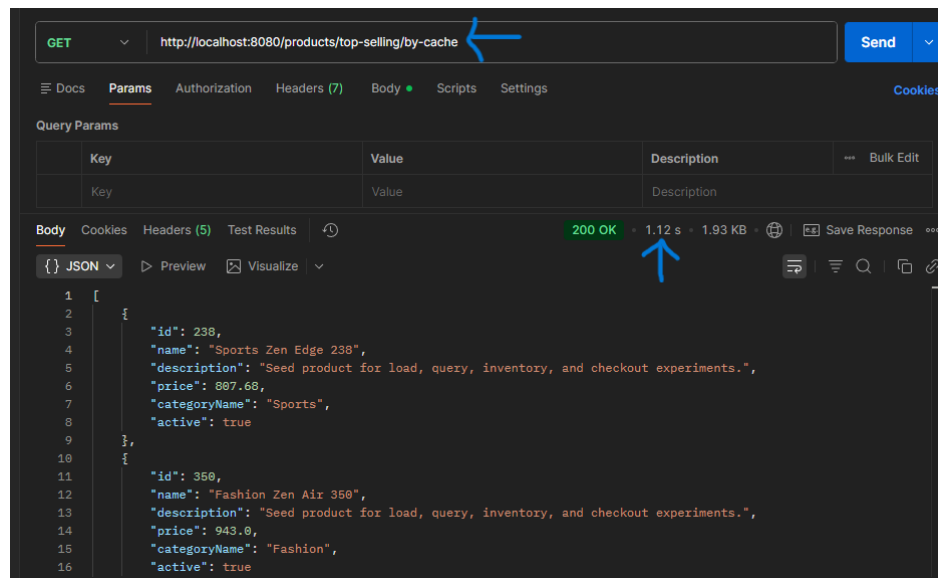
Query Params

Key	Value	Description
Key	Value	Description

Body: 200 OK • 553 ms • 379 B

```
1 {
2   "name": "ecommerce.cache.hit",
3   "measurements": [
4     {
5       "statistic": "COUNT",
6       "value": 5470.0
7     }
8   ],
9   "availableTags": [
10    {
11      "tag": "cache",
12      "values": [
13        "topProducts"
14      ]
15    }
16  ]
17 }
```

يدل الـ **cache hit** الأعلى: على عدد الـ **cache hit** التي تمت بالـ **redis**، وهي **5470**.



بعد قيامنا بتعديل منتج من المنتجات الموجودة لدينا، تمت عملية الـ cache invalidation، وعند اول طلب للـ **topProducts** بعد التعديل، استغرق 1 ثانية، حيث هذا يدل على الاتصال بقاعدة البيانات (miss cache).

خاتمة ما قمنا به:

تم تطبيق Redis كطبقة Distributed Cache مشتركة بين جميع نسخ التطبيق الموجودة خلف Load Balancer. الهدف من ذلك هو تقليل عدد الاستعلامات المتكررة إلى قاعدة البيانات في عمليات القراءة المتكررة مثل عرض تفاصيل المنتج، قائمة المنتجات، المنتجات الأكثر مبيعاً، وتقارير المبيعات اليومية.

تم اعتماد نمط Cache-Aside، حيث تحاول الخدمة قراءة البيانات من Redis أولاً، وفي حال عدم وجودها يتم الرجوع إلى قاعدة البيانات ثم تخزين النتيجة في Redis مع TTL مناسب لنوع البيانات. تم ضبط TTL مختلف لكل cache؛ فبيانات تفاصيل المنتج لها TTL متوسط، بينما المنتجات الأكثر مبيعاً لها TTL قصير لأنها تتغير بشكل أسرع.

لم يتم استخدام Redis كمصدر حقيقة للبيانات الحرجة مثل المخزون أو الدفع. قرار إنشاء الطلب وتخفيض المخزون يتم دائماً داخل معاملة قاعدة البيانات باستخدام آليات التزامن المناسبة، لأن الاعتماد على stock مخزن مؤقتاً قد يؤدي إلى overselling عند وجود طلبات متزامنة.

عند تعديل منتج، يتم حذف الكاش المرتبط به باستخدام Cache Eviction حتى لا يستمر النظام بإرجاع بيانات قديمة. كما تم حذف قائمة **topProducts** نتيجة لذلك. وتمت مراقبة أداء هذه الطبقة باستخدام قياسات قبل وبعد تشمل response time و p95 وعدد استعلامات قاعدة البيانات و cache hit.

الطلب السابع للتحكم في الأقفال الموزعة:

في الطلب الأول درسنا:

- Optimistic Locking
- Pessimistic Locking

حيث قمنا بالمقارنة ما بينهم وقمنا بتطبيق Pessimistic Locking في الأخير وذلك بسبب حالة التخزين الحرجة التي تتطلب عدم التداخل بشكل قوي، حيث كان التركيز على حماية قيمة stock لسجل داخل قاعدة البيانات.

القفل هنا كان على مستوى السجل في DB، ولكن كيف سنمنع أكثر من application instance أو scheduler أو worker من تنفيذ نفس العملية الحرجة في نفس الوقت، عندما يكون النظام موزعاً على أكثر من خادم؟

هنا تظهر لدينا مشكلة هندسية جديدة:

حيث بعد الطلب الخامس أصبح لدينا أكثر من نسخة للتطبيق، وكل نسخة لديها نفس الكود، هذا يعني أن أي شيء موجود داخل التطبيق سيتكرر في كل instance:

- @Scheduled job
- cache rebuild
- batch launcher
- background cleanup
- coupon processor

مثلاً في الطلب الرابع قمنا بإضافة scheduler، ونحن لدينا ثلاثة نسخ من التطبيق، فهذا الـ scheduler سيعمل في App 1، App 2، App 3، والنتيجة المحتملة:

- DailySalesJob يعمل 3 مرات لنفس اليوم
- التقرير يتكرر
- staging table تتلوث
- summary table تُعاد كتابتها
- ضغط زائد على DB
- نتائج غير متوقعة

هنا سنحتاج إضافة Distributed Lock

Distributed Lock:

Distributed Lock هو قفل مشترك بين أكثر من process أو JVM أو server، أي بدلاً من أن يكون القفل في تطبيق واحد يصبح محفوظاً في مكان تخزين مشترك بين نسخ التطبيق، مثل:

- Redis

- Database
- ZooKeeper
- Etcd
- Consul

حيث تكون فكرة القفل أنّ 1 App يحاول أخذ "lock: daily-sales-job:2026-06-13" على سبيل المثال، و 2 App يحاول أخذ نفس lock، و 3 App يحاول أخذ نفس lock. وواحد فقط ينجح والبقية إما تنتظر أو تتخطى التنفيذ.

حيث يلعب هنا **distributed locks** كآلية مفيدة عندما تحتاج عمليات مختلفة إلى العمل على موارد مشتركة بطريقة **mutually exclusive**، أي بشكل يمنع أكثر من عميل من امتلاك القفل لنفس المورد في نفس اللحظة.

المفاهيم التي نحتاج تطبيقها عند استخدام أي Distributed Lock:

أي **Distributed Lock** لا يكفي أن يقول لنا: واحد فقط دخل. بل نحتاج لكي نقوم بتطبيق القفل المشترك بشكل وثيق، ان نضمن تنفيذ المفاهيم التالية:

- **Mutual Exclusion**: في اللحظة نفسها يجب أن يمتلك القفل منفذ واحد فقط لنفس المورد المنطقي.
- **TTL أو Lease Time**: يجب ألا يبقى القفل إلى الأبد إذا مات الـ instance التي أخذته.
- **Ownership Token**: الذي يأخذ القفل يجب أن يكون هو فقط القادر على تحريره، حتى لا يحذف قفل instance أخرى.
- **fe UnlockSa**: لا يتم حذف القفل إلا إذا كان ما يزال مملوكاً لنفس الـ owner.
- **Idempotency كخط دفاع**: حتى لو فشل القفل أو انتهى قبل الوقت، يجب ألا تؤدي إعادة التنفيذ إلى فساد البيانات.
- **Monitoring**: يجب أن نستطيع قياس عدد مرات أخذ القفل، وعدد مرات التخطي، وزمن الانتظار، وعدد حالات الفشل.

Use Cases التي سنطبقها في هذا الطلب:

لن نقوم بإضافة حالات جديدة حيث سنطور على الحالات التي قمنا بها في الطلبات الماضية:

- في الطلب الرابع عندما قمنا بتطبيق الـ Scheduler مع الـ Spring Batch، حيث بعد إضافة Load Balancer في الطلب الخامس اضطررنا ان نقوم بتفعيل المجدول على App1 فقط، ولكن هذه الحالة ليست بحالة كافية وذلك عند تعطل هذه النسخة من التطبيق سيؤدي هذا الى توقف عملية المجدول وهنا يتوجب علينا تفعيل المجدول على أكثر من instance وتطبيق القفل المشترك لضمان تنفيذه من قبل instance واحد في نفس الوقت.

- في الطلب السادس عند تخزين المنتجات الأعلى مبيعاً في الكاش، ذكرنا مفهوم الـ Cache stampede حيث قمنا بتطبيق قفلاً بسيطاً على عملية الـ Cache Rebuild، ولكن لم نقوم بتطبيق مفاهيم القفل المشترك بشكل كافٍ ويتوجب علينا العودة هنا لهذه الحالة وتطبيق مفهوم القفل المشترك بشكل وثيق.

سنقوم بدراسة البدائل قبل الاختيار:

قبل الوصول إلى الحل النهائي، تمت دراسة عدة بدائل. المهم هنا ألا نحكم على البديل بأنه جيد أو سيئ بشكل مطلق، بل سنحدد أين يكون مناسباً وأين لا يناسب مشروعنا.

1. استخدام synchronized أو Java Lock داخل التطبيق

هذا **الحل مناسب** عندما تكون المشكلة داخل JVM واحدة فقط، مثل حماية Map داخلية أو منع Threadين داخل نفس التطبيق من تنفيذ كود محلي في نفس اللحظة.

- **ميزته** أنه بسيط وسريع ولا يحتاج Redis أو Database.
- **لكنه غير مناسب** لحالتنا لأن النظام أصبح موزعاً على أكثر من instance. القفل داخل 1App لا تعرف عنه 2App شيئاً، والقفل داخل 2App لا تعرف عنه 3App شيئاً. لذلك synchronized يمنع التزامن المحلي فقط، لكنه لا يحل مشكلة Scheduler يعمل في ثلاث نسخ مختلفة.

2. استخدام Database Row Lock

- هذا **الحل مناسب** عندما يكون المورد المحمي صفّاً واضحاً داخل قاعدة البيانات. مثلاً حماية stock لمنتج معين أو حماية row خاص بـ coupon. في هذه الحالة يمكننا استخدام SELECT FOR UPDATE أو optimistic locking أو atomic update.
- **لكنه ليس الخيار الطبيعي** لتنسيق عملية عامة مثل DailySalesJob أو cache rebuild، لأن هذه العمليات ليست دائماً مرتبطة بصف واحد واضح. يمكن بناء جدول lock داخل DB، لكن عندها نكون قد انتقلنا عملياً إلى based distributed lock-DB، وهذا هو ما سنستخدمه للـ Scheduler عبر ShedLock.

3. الاعتماد على Unique Constraints و Idempotency فقط

في كثير من الحالات لا نحتاج Distributed Lock إذا كانت قاعدة البيانات تمنع التكرار من الأصل. مثلاً يمكن منع إنشاء فاتورتين لنفس order عبر order (id_order) UNIQUE، ويمكن منع معالجة نفس event مرتين عبر event (name_consumer, id_event) message_processed.

لكن في حالة جدولة الـ **DailySalesJob**، حتى لو كانت النتائج النهائية محمية بقيود فريدة، فإن تشغيل الـ **Job** ثلاث مرات سيستهلك الـ **DB** و **CPU** وقد يسبب ضغطاً كبيراً. لذلك سنستخدم **Distributed Lock** لمنع التكرار من البداية، وسنبقي **idempotency** و **unique constraints** كخط دفاع إضافي، يمكن تطبيقه في المستقبل.

4. Queue Serialization

فكرة هذا الحل أن نرسل العمل إلى **Queue** واحدة ويقوم **Consumer** واحد بمعالجته بالتتابع. هذا مناسب جداً للمهام غير المتزامنة التي تحتاج ترتيباً واضحاً، مثل معالجة أوامر خلفية أو رسائل **Event-driven**.

لكنه ليس الاختيار الأبسط لحماية **Scheduled Batch** في مشروعنا، لأن لدينا أصلاً **Spring Scheduler** و **Spring Batch**. إدخال **Queue** فقط لمنع **Scheduler** من التكرار سيزيد التعقيد. لذلك سيكون **ShedLock** أكثر مباشرة. أما في حالات **payment** أو **notification** فبنية **Queue** مناسبة أكثر، وقد درسناها في الطلب الثالث.

5. Redis SET NX PX

هذا هو النمط الأساسي للقفل باستخدام **Redis**. الفكرة هي أن نحاول إنشاء مفتاح **lock** فقط إذا لم يكن موجوداً، مع تحديد **TTL**، وبقيمة **random token** تمثل مالك القفل. الشكل العام:

وعند التحرير يجب ألا نحذف القفل مباشرة، بل نستخدم **Lua script** يحذف المفتاح فقط إذا كانت قيمته تساوي الـ **token** الخاص بنا. هذا يمنع أن يقوم **request** قديم بحذف قفل جديد أخذته **instance** أخرى بعد انتهاء الـ **TTL**.

- هذا الحل مناسب للعمليات القصيرة مثل منع **cache stampede** أو إعادة بناء كاش معين.
- لكنه أقل ملاءمة للـ **Scheduled Job** الطويل، وأقل راحة من **Redisson RLock** من ناحية إدارة **tryLock** و **leaseTime** و **unlock**.

6. Redlock (الخوارزمية التي تم ذكرها في المحاضرة)

Redlock هي خوارزمية متقدمة من **Redis** تهدف إلى جعل القفل أكثر تحملاً لفشل **Redis instance** واحدة. بدلاً من الاعتماد على **Redis** واحد، يتم استخدام عدة **masters Redis** مستقلة غالباً يكونوا 5، ويحاول العميل أخذ القفل من أغلبية العقد خلال زمن أقل من مدة صلاحية القفل. إذا نجح في الأغلبية يعتبر أنه امتلاك القفل.

- تكون **Redlock** مناسبة عندما يكون لدينا بيئة موزعة أكبر ونريد تقليل خطر الاعتماد على **Redis** واحد.

- **لكنها تزيد التعقيد التشغيلي**، كما أنها لا يجب أن تكون الضمان الوحيد للبيانات المالية أو المخزون. لذلك لم نختارها في التطبيق العملي هنا، لأن حالتنا الاستخدام لدينا لا تحتاجان هذا المستوى من التعقيد.

7. ShedLock (أحد الحلول المختارة)

ShedLock **مناسب** جداً للـ Scheduled Tasks في الأنظمة التي تعمل على عدة instances/nodes. فكرته أنه عند دخول وقت الـ Scheduler، تحاول كل instance أخذ lock من مصدر مشترك. واحدة فقط تنجح، والباقي تتخطى التنفيذ. لذلك هو ليس Scheduler بحد ذاته، بل طبقة قفل تمنع تكرار تنفيذ نفس المهمة المجدولة.

هذا هو **الحل الذي سنختاره لحالة** المجدول لدينا، وسنستخدم JDBC provider بدلاً من Redis provider لأن الـ atchB مرتبط أصلاً بقاعدة البيانات و Spring Batch JobRepository.

8. Redisson RLock (أحد الحلول المختارة)

Redisson RLock يقدم API يشبه Java Lock لكنه موزع عبر Redis. **ميزته:**

- أنه يدعم tryLock مع waitTime و leaseTime.
- ويمنع unlock من thread غير المالك.
- ويدعم watchdog لتمديد القفل إذا لم نحدد leaseTime.

لذلك هو مناسب لحالات قصيرة داخل request flow مثل cache rebuild أو منع cache stampede.

سنختاره لحالة topProducts cache rebuild لأن العملية مرتبطة أصلاً بـ Redis والكاش، ولأننا نحتاج انتظاراً قصيراً جداً فقط، وليس قفلاً طويلاً مثل الـ Batch، كما أننا **سنفضله على Redis SET NX PX** بسبب الميزات التي يقدمها.

القرار النهائي في مشروعنا

لن نستخدم أداة واحدة لكل حالات الأقفال الموزعة، لأن طبيعة الخطر مختلفة. لذلك سيكون القرار كالتالي:

سبب الاختيار	الحل المختار	الخطر	حالة الاستخدام
مصمم للـ Scheduled Tasks ويستخدم DB مشتركة	+ShedLock JDBC	تشغيل نفس الـ Batch من أكثر من instance	DailySalesJob Scheduler
قفل قصير حول rebuild مع tryLock و leaseTime	Redisson RLock	Cache stampede وضغط مفاجئ على DB	/Cache Rebuild Top Products

حالة الاستخدام الأولى: حماية DailySalesJob Scheduler

في الطلب الرابع قمنا ببناء DailySalesJob لمعالجة المبيعات اليومية على شكل Batch. هذا الـ Job يُشغل غالباً بعد منتصف الليل لمعالجة مبيعات اليوم السابق. لكن بعد تشغيل أكثر من instance للتطبيق، يصبح كل Scheduler داخل كل instance قادراً على تشغيل نفس الـ Job. لذلك نحتاج قفلاً موزعاً يمنع هذا التكرار

السيناريو بدون Distributed Lock

إذا كان لدينا ثلاث نسخ من التطبيق، وكان الـ cron مضبوطاً على 00:30، فستحاول النسخ الثلاث تشغيل نفس الـ Job:

- 00:30:00 App1 starts DailySalesJob for 2026-06-13
- 00:30:00 App2 starts DailySalesJob for 2026-06-13
- 00:30:00 App3 starts DailySalesJob for 2026-06-13

هذا يؤدي إلى تكرار العمل، ضغط زائد على قاعدة البيانات، وربما تداخل في جداول staging أو إعادة كتابة النتيجة أكثر من مرة. حتى لو كان الـ Writer مصمم بطريقة idempotent، فإن النظام سيستهلك موارد لا داعي لها.

التصميم الذي سنعتمده مع ShedLock

سنستخدم جدول مشترك باسم shedlock داخل قاعدة البيانات. عندما يحين وقت تشغيل الـ Job، تحاول كل instance أخذ القفل باسم ثابت مثل scheduler-obj-sales-daily. النسخة التي تنجح تشغل الـ Job، أما النسخ الأخرى فتتخطى التنفيذ. وشكل الجدول سيكون:

```
CREATE TABLE shedlock (  
    name VARCHAR(64) NOT NULL,  
    lock_until TIMESTAMP(3) NOT NULL,  
    locked_at TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),  
    locked_by VARCHAR(255) NOT NULL,  
    PRIMARY KEY (name)  
);
```

معنى الأعمدة:

- name هو اسم القفل.
- lock_until يحدد إلى متى يبقى القفل فعالاً.
- locked_by يوضح أي instance أخذت القفل. هذا يساعدنا أيضاً في المراقبة والتوثيق.

إعدادات القفل ستكون:

- **lockAtMostFor**: الحد الأعلى لمدة القفل، بحيث سيكون أطول من أسوأ زمن متوقع لل Batch. مثلاً **H2PT** (ساعتين)، إذا كان الـ Job قد يستغرق أقل من ساعتين عادة.
- **lockAtLeastFor**: الحد الأدنى لمدة الاحتفاظ بالقفل، ويمنع إعادة تشغيل نفس المهمة بسرعة كبيرة إذا انتهت بسرعة غير متوقعة.
- **usingDbTime**: الاعتماد على وقت قاعدة البيانات بدلاً من وقت كل instance لتقليل مشاكل اختلاف الساعات بين الخوادم.
- **عدم إضافة id.run** في JobParameters في تشغيل الـ Scheduler، حتى لا يتم إنشاء JobInstance جديد لنفس salesDate كل مرة. بحيث إذا استخدمنا **id.run** عشوائي في التشغيل المجدول، فقد نسمح بتشغيل نفس اليوم كـ JobInstance جديد أكثر من مرة. لذلك في التشغيل الإنتاجي للـ Scheduler سنمرر salesDate فقط، أما id.run فيمكن استخدامه في التجارب اليدوية عند الحاجة.

ما سنقوم به في الكود

- إضافة dependencies الخاصة بـ spring-shedlock و jdbc-provider-shedlock-template.
- إنشاء جدول shedlock.
- إضافة ShedLockConfig لتفعيل @EnableSchedulerLock وتسجيل JdbcTemplateLockProvider.
- إضافة SchedulerLock@ على تابع تشغيل DailySalesJob.
- ضبط lockAtMostFor و lockAtLeastFor بناءً على زمن الـ Batch الفعلي.
- التأكد أن JobLauncher يعمل بشكل synchronous أثناء الـ Scheduler حتى لا يتم تحرير القفل قبل انتهاء الـ Job.

حالة الاستخدام الثانية: Rebuild Lock Cache

في الطلب السادس قمنا بتطبيق Redis Cache على المنتجات الأكثر مبيعاً topProducts. هذه البيانات تظهر عادة في الصفحة الرئيسية أو slider، وبالتالي يتم طلبها كثيراً جداً. عند انتهاء TTL، لو وصل عدد كبير من المستخدمين في نفس اللحظة، سيرى الجميع cache miss وسيحاول الجميع إعادة تحميل البيانات من قاعدة البيانات. هذه المشكلة تسمى Cache Stampede أو Thundering Herd، حيث تم تذكرها هناك بشكل مبسط.

السيناريو بدون Lock

1. top-products TTL expired
2. 100 users request /products/top-selling at the same time
3. 100 cache misses
4. 100 DB queries
5. 100 rebuild attempts
6. DB spike + high p95/p99

في هذه الحالة لا تكون المشكلة فساد بيانات، بل انفجار مفاجئ في الضغط على قاعدة البيانات. لذلك نحتاج قفلاً قصيراً حول عملية rebuild، وليس قفلاً طويلاً مثل قفل الـ Batch.

لماذا قمنا بتفضيل Redisson RLock على SET NX PX ؟

يمكننا كتابة Redis lock يدوياً باستخدام SET NX PX مع token و Lua unlock، لكن Redisson RLock يوفر لنا **مميزات**:

- واجهة أعلى مستوى وأسهل في الكود.
- نستطيع استخدام tryLock لتحديد مدة انتظار قصيرة.
- و leaseTime لتحرير القفل تلقائياً إذا حدث خطأ.
- كما أن Redisson يمنع unlock من thread لا يملك القفل.

لذلك سنستخدم **Redisson RLock** في حالة **cache rebuild** لأنها عملية قصيرة، مرتبطة بالكاش، وتحتاج حماية من stampede أكثر من حاجتها إلى ضمانات transactions.

التصميم القوي الذي سنستخدمه للـ Cache Rebuild

بدلاً من مفتاح واحد فقط للكاش، سنستخدم **مفتاحين**:

- **Fresh key**: يحمل النسخة الحديثة بصلاحية قصيرة، مثل 10:limit:products-top.
- **Stale key**: يحمل نسخة أقدم قليلاً بصلاحية أطول، مثل 10:limit:stale:products-top.

الفكرة من **تصميمنا** أن المستخدم إذا لم يحصل على fresh value ولم يستطع أخذ القفل لإعادة البناء، يمكنه الحصول على stale value مقبولة لفترة قصيرة بدلاً من الانتظار الطويل أو ضرب قاعدة البيانات.

تدفق التنفيذ سيكون:

1. يطلب المستخدم قائمة المنتجات الأعلى مبيعاً.
2. الخدمة تبحث عن freshKey في Redis.
3. إذا وجدته، ترجع البيانات مباشرة وتُحسب Cache Hit.

4. إذا لم تجده، تحاول أخذ Redisson RLock على products-top:rebuild-cache:lock.
5. إذا أخذت القفل، تعمل check-double لأن request آخر ربما بنى الكاش قبلها.
6. إذا ما زال الكاش فارغاً، تقرأ من DB، وتخزن freshKey و staleKey، ثم تحرر القفل.
7. إذا لم تأخذ القفل، تحاول إرجاع staleKey، أو تنتظر مدة قصيرة ثم تقرأ freshKey.
8. إذا لم يوجد stale ولا fresh، يكون fallback الأخير هو قراءة DB، ولكن بدون إعادة بناء الكاش إذا لم نملك القفل.

لماذا نحتاج Double Check ؟

قد يحدث أن Request A يرى cache miss ويأخذ القفل ويعيد بناء الكاش. في نفس الوقت Request B كان ينتظر القفل. بعد أن ينتهي A ويحرر القفل، يأخذ B القفل. بدون check-double سيعيد B بناء نفس الكاش مرة ثانية. لذلك بعد أخذ القفل يجب أن نقرأ freshKey مرة أخرى. إذا أصبح موجوداً، نرجعه مباشرة بدون DB query جديدة.

اختيار waitTime و leaseTime

في حالة الاستخدام لدينا لا نريد أن ينتظر المستخدم طويلاً. لذلك نختار waitTime قصير جداً مثل 100-200 ms. أما leaseTime فيجب أن يكون أطول من زمن إعادة البناء المتوقع. مثلاً إذا query إعادة بناء topProducts تأخذ أقل من ثانية عادة، يمكن اختيار leaseTime = 10s كشبكة أمان. المهم ليس الرقم نفسه، بل المنطق: waitTime قصير حتى لا نبطئ المستخدمين، و leaseTime كافٍ حتى لا ينتهي القفل أثناء عملية rebuild.

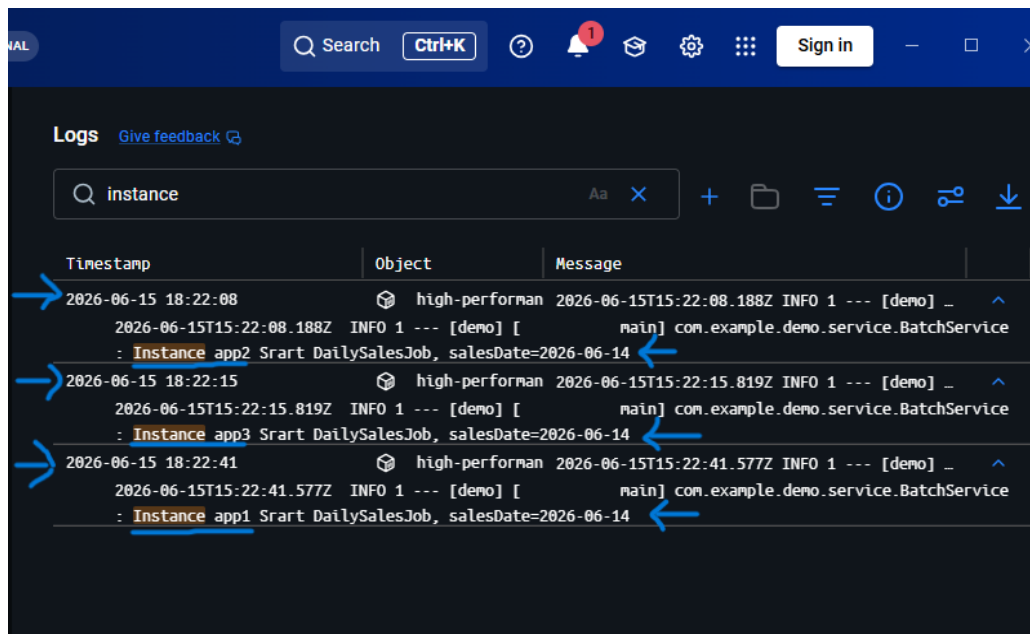
ما سنقوم به في الكود

- إضافة encyRedisson depend وتكوين RedissonClient.
- تعريف مفاتيح freshKey و staleKey و lockKey للـ topProducts.
- استخدام tryLock.RLock مع waitTime قصير و leaseTime مناسب.
- تطبيق check-double بعد أخذ القفل.
- تخزين freshKey بـTTL قصير مع jitter لتقليل انتهاء المفاتيح في نفس اللحظة.
- تخزين staleKey بـTTL أطول ليكون fallback عند stampede.
- تسجيل metrics: cache hit , cache miss , lock acquired , lock skipped , stale , served , DB fallback.
- حذف freshKey و staleKey عند تحديث منتج ما.

المقاييس النهائية:

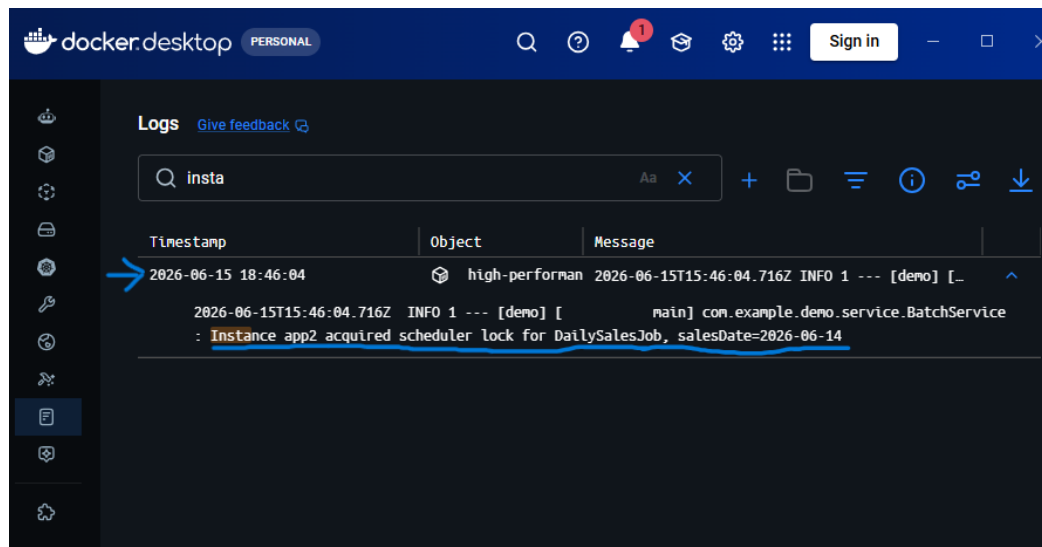
1- ShedLock Use Case :

- قبل وضع الـ shedlock



ما تدل عليه الـ logs الظاهرة في الصورة، ان كل نسخة من الثلاث نسخ قامت بتنفيذ الـ job في نفس الوقت، ما أدى الى تكرار النتائج نفسها.

- بعد وضع الـ shedlock:



حيث تدل الصورة على انه بعد إضافة الـ shedlock، اول instance قام بتنفيذ الـ job وضع قفل على الـ scheduler، لمنع النسخ الأخرى من إعادة تنفيذه وجعلهم ان يقوموا بتخطيه عند وجود القفل عليه، حيث قام بتخزين هذا القفل في جدول الـ shedlock والصورة التالية تدل على هذا:

```
mysql> select * from shedlock;
+-----+-----+-----+-----+
| name                | lock_until                | locked_at                | locked_by                |
+-----+-----+-----+-----+
| daily-sales-job-scheduler | 2026-06-15 15:46:50.490 | 2026-06-15 15:45:50.490 | 6627705b6de4            |
+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql>
```

Redisson Cache Stampede Use Case -2

- قبل وضع قفل redisson:

- بعد وضع القفل:

خاتمة ما قمنا به في الطلب السابع

تم في هذا الطلب نقل مفهوم الأقفال من مستوى سجل داخل قاعدة البيانات إلى مستوى النظام الموزع. بعد تشغيل عدة نسخ من التطبيق خلف **Load Balancer**، أصبحت المشكلة ليست فقط أن مستخدمي stock، بل أن أكثر من instance قد تنفذ نفس المهمة الخلفية أو تعيد بناء نفس الكاش في نفس اللحظة.

تم اختيار حالتنا استخدام واضحتين: الأولى هي حماية DailySalesJob الذي يتقاطع مع Batch Processing في الطلب الرابع، والثانية هي حماية cache rebuild للمنتجات الأكثر مبيعاً والتي تتقاطع مع Distributed Cache في الطلب السادس. في الحالة الأولى تم اختيار ShedLock مع JDBC لأنه مصمم للمهام المجدولة ويمنع تشغيل نفس task على أكثر من node. وفي الحالة الثانية تم اختيار Redisson RLock لأنه مناسب لقفل قصير داخل request path لمنع cache stampede.

بهذا يكون **الطلب السابع** قد أثبت فهماً هندسياً عميقاً: Distributed Lock هو أداة تنسيق بين nodes، وليس بديلاً عن سلامة المعاملات أو قيود قاعدة البيانات. نستخدمه عندما تكون المشكلة ناتجة عن تعدد نسخ التطبيق، ونختار أدواته حسب طبيعة العملية: ShedLock لل schedulers، و Redisson RLock لل cache rebuild، و raintsDB const للبيانات الحرجة.

الطلب الثامن: سلامة المعاملات Transaction Integrity / ACID

في نظام التجارة الإلكترونية لا تكون العملية التجارية عبارة عن تعديل جدول واحد فقط، بل غالباً تكون عبارة عن سلسلة من التغييرات المترابطة على عدة جداول. على سبيل المثال، عندما يقوم المستخدم بإنشاء طلب شراء، فإن النظام لا ينشئ فقط سجل **order**، بل يجب أن يتحقق من المخزون، ينقص كمية المنتج، ينشئ طلب، يسجل محاولة الدفع بشكل مبسط، وربما ينشئ حدثاً داخلياً لمعالجة الفاتورة أو الإشعار لاحقاً.

المشكلة هنا أنه إذا تمت بعض هذه الخطوات وحدث **فشل** قبل اكتمال الباقي، فقد يدخل النظام في حالة غير صحيحة. مثلاً يمكن أن ينقص المخزون، لكن لا يتم إنشاء الطلب. أو يمكن أن يتم إنشاء الطلب بدون عناصر الطلب. أو يمكن أن يتم تسجيل محاولة دفع بدون وجود طلب صحيح مرتبط بها. لذلك نحتاج إلى مفهوم **Transaction Integrity**، أي أن العملية التجارية يجب أن تنجح بالكامل أو تفشل بالكامل، دون ترك أثر جزئي يفسد حالة النظام.

هذا الطلب يختلف عن الطلب الأول والطلب السابع. في الطلب الأول كان التركيز على مشكلة التزامن عند تعديل نفس **stock** من أكثر من مستخدم في نفس الوقت، وقمنا بدراسة **optimistic locking** و **pessimistic locking** لحماية سجل داخل قاعدة البيانات. أما في الطلب السابع فقد انتقلنا إلى مستوى النظام الموزع، حيث احتجنا إلى **lock distributed** لمنع أكثر من **instance** من تشغيل نفس الـ **scheduler** أو إعادة بناء نفس الكاش في نفس اللحظة.

أما هنا في الطلب الثامن فيركّز على **وحدة العمل** نفسها: هل جميع التغييرات التي تنتمي إلى عملية **checkout** واحدة ويتم حفظها أو التراجع عنها كوحدة واحدة؟ بمعنى آخر، القفل يمنع التداخل، أما **transaction** فتمنع الحالة الجزئية.

أهمية **Transaction Integrity** في مشروعنا
عملية إنشاء طلب شراء في مشروعنا يمكن أن تتضمن:

1. التحقق من المنتج.
2. التحقق من أن الكمية المطلوبة موجبة.
3. إنقاص المخزون.
4. إنشاء **order**.
5. إنشاء **order_item**.
6. حفظ **snapshot** من اسم وسعر المنتج وقت الشراء.

لو لم تكن هذه الخطوات داخل **transaction** صحيحة، فقد تظهر مشاكل خطيرة:
• **Stock** ينقص رغم فشل الطلب.

- **order** يُحفظ بدون **order_items**.
 - **Event** معين يتم إنشاؤه رغم فشل العملية التجارية.
 - المستخدم يرى أن الطلب فشل، بينما قاعدة البيانات تغيرت جزئياً.
- لذلك الهدف في هذا الطلب ليس فقط وضع **@Transactional**، بل تحديد مكان المعاملة وحدودها، ومعرفة ما الذي يجب أن يدخل داخل **transaction** وما الذي يجب أن يبقى خارجها.

تعتمد سلامة المعاملات على مفهوم **ACID**:

- **Atomicity**:
تعني أن كل خطوات العملية تنجح معاً أو تفشل معاً. في **checkout**، إنقاص المخزون وإنشاء الطلب وعناصر الطلب ومحاولة الدفع يجب أن تكون وحدة واحدة. إذا فشلت خطوة بعد إنقاص المخزون، يجب أن يعود المخزون كما كان.
- **Consistency**:
تعني أن **transaction** تنتقل قاعدة البيانات من حالة صحيحة إلى حالة صحيحة أخرى. مثلاً لا يجب أن يصبح **stock** سالباً، ولا يجب أن يوجد **order_item** بدون **order**، ولا يجب أن يوجد هناك محاولة دفع خارجية بدون وجود **order**.
- **Isolation**:
تعني كيف تتفاعل **transaction** مع **transactions** أخرى تعمل في نفس الوقت. هذا مهم جداً عند وجود أكثر من مستخدم يحاول شراء نفس المنتج. لذلك لا يكفي وجود **@Transactional** فقط، بل نحتاج أيضاً إلى آلية صحيحة لحماية المخزون مثل **atomic update** أو **pessimistic locking**.
- **Durability**:
تعني أنه بعد نجاح **commit**، تصبح التغييرات محفوظة في قاعدة البيانات. فإذا تم إنشاء **order** وعناصر أخرى بنجاح، يجب أن تبقى موجودة بعد **commit**.

دراسة البدائل قبل اختيار الحل

قبل الوصول إلى التصميم النهائي، تمت دراسة عدة بدائل. الهدف هنا ليس أن نقول إن البديل جيد أو سيئ بشكل مطلق، بل أن نحدد أين يكون مناسباً وأين لا يناسب مشروعنا.

1. عدم استخدام **Transaction** واضحة

هذا الحل قد يكون مقبولاً في عمليات قراءة بسيطة أو في عمليات لا تعدّل أكثر من جدول. مثلاً **GET /products** أو **GET /categories** لا تحتاج غالباً إلى **transaction** معقدة لأنها عمليات قراءة فقط.

لكنه غير مناسب لعملية **checkout**، لأن **checkout** يغيّر عدة جداول مترابطة. لو حدث فشل بعد تحديث **stock** وقبل إنشاء **order**، ستبقى قاعدة البيانات في حالة غير صحيحة.

لذلك عدم استخدام transaction يمكن أن يكون مناسباً للقراءات البسيطة، لكنه **مرفوض** للعمليات التجارية المركبة مثل إنشاء الطلب.

2. وضع Transaction على كل Repository Method فقط

قد يبدو هذا الحل منظماً، حيث كل repository operation تكون transaction مستقلة. مثلاً decreaseStock داخل transaction، ثم createOrder داخل transaction أخرى.

لكن هذا خطير في حالتنا. إذا نجحت transaction الخاصة بإنقاص المخزون، ثم فشلت transaction الخاصة بإنشاء الطلب، فسيتم حفظ نقص المخزون رغم عدم وجود **order**. أي أن كل عملية صغيرة نجحت وحدها، لكن العملية التجارية ككل فشلت. لذلك هذا الأسلوب **مناسب لعمليات بسيطة ومستقلة**، لكنه **غير مناسب** عندما تكون لدينا وحدة عمل تجارية واحدة تمتد عبر عدة جداول.

3. وضع @Transactional على Controller

هذا ممكن تقنياً، لكنه ليس تصميمياً جيداً Controller. مسؤول عن استقبال HTTP request وتحويله إلى response، وليس مسؤولاً عن تحديد حدود العملية التجارية. كما أن وضع transaction في Controller قد يجعل المعاملة واسعة جداً وتبقى مفتوحة أثناء عمليات لا يجب أن تكون داخل transaction.

لذلك المكان الأفضل لل transaction هو Application Service أو Use Case Service، مثل CheckoutService، لأن هذا هو المكان الذي يمثل العملية التجارية الكاملة.

4. وضع Transaction واسعة جداً مثلاً:

```
@Transactional
public void checkout() {
    decreaseStock();
    createOrder();
    callExternalPaymentSystem();
    sendEmail();
    generateInvoicePdf();
}
```

هذا التصميم غير مناسب، لأن transaction ستبقى مفتوحة أثناء عمليات خارجية وبطيئة مثل إرسال email أو استدعاء نظام دفع خارجي أو إنشاء PDF. هذا يؤدي إلى حجز DB connection لمدة طويلة، وزيادة زمن الأقفال، وارتفاع p95 و p99، وربما حدوث timeouts، وأفضل مثال على ذلك هو تطبيق ذلك في الطلب الثالث عندما تم إنشاء التابع

buyWithAsync() داخل ProductService، حيث سنقوم بتصحيح هذا الأمر في هذا الطلب.

وبالتالي **transaction** يجب أن تكون قصيرة ومحددة، وتحتوي فقط على تغييرات قاعدة البيانات التي يجب أن تكون **atomic**.

5. الاعتماد على Isolation Level فقط

رفع isolation level إلى **SERIALIZABLE** قد يبدو حلاً سهلاً، لكنه ليس مناسباً كحل عام لكل النظام. صحيح أنه يعطي عزلاً أقوى، لكنه قد يقلل throughput ويزيد lock waiting وdeadlocks.

في مشروعنا لا نعتمد على isolation وحده لحماية المخزون، بل نستخدم atomic update عند إنقاص stock. بهذه الطريقة تكون عملية التحقق من المخزون وتخفيضه خطوة واحدة داخل قاعدة البيانات، مما يقلل خطر **overselling** و **lost update** دون رفع كل النظام إلى **SERIALIZABLE**.

القرار النهائي في مشروعنا

القرار النهائي هو استخدام:

- @Transactional على تابع **placeOrder** الجديد داخل **ProductService**.
- Atomic update لتخفيض المخزون، حيث لن نعتمد هنا على القفل التبادلي كما كنا نقوم به في الطلبات الأولى، وذلك لرفع سرعة استجابة الطلبات وزيادة الأداء التوازي.
- إنشاء **order_items** و **order** داخل نفس **transaction**.
- استخدام خدمة الدفع الخارجي عن طريق **RabbitMQ** التي كنا نستخدمها في الطلب الثالث، مع جعلها خارج المعاملة الرئيسية، أي تدارك الخطأ الذي وقعنا به مسبقاً، لأن فشل ال **rabbitMQ** في أداء خدمته سيُفشل كامل المعاملة، كما أن فشل المعاملة بعد تجاوز سطر تنفيذه، سيؤدي إلى إرسال رسالة دفع دون اكتمال عملية الطلب الحقيقية..
- استخدام **Failure Scenarios** لإثبات أن **rollback** يعمل بشكل صحيح.

حالة الاستخدام الأساسية Checkout Transaction:

حالة الاستخدام التي سنطبق عليها **Transaction Integrity** هي عملية **checkout**، لأنها أقوى حالة في مشروع التجارة الإلكترونية، وتجمع بين المخزون والطلب والدفع المحاكى والحدث الداخلي. السيناريو:

المستخدم يرسل طلب شراء يحتوي على:

- **userId**
- **productId**
- **quantity**
- **paymentMethod**

بعد ذلك يجب أن يقوم النظام بالخطوات التالية داخل **transaction** واحدة:

1. التحقق من أن الكمية موجبة.
 2. إنقاص stock بشكل atomic.
 3. قراءة بيانات المنتج من أجل snapshot.
 4. إنشاء order بحالة PENDING_PAYMENT أو CREATED.
 5. إنشاء order_item.
 6. القيام بعملية الدفع، حيث يتم خصم الرصيد من حساب المستخدم، مع تنفيذ خدمة الـ RabbitMQ، خارج المعاملة.
 7. تنفيذ commit.
- إذا فشلت أي خطوة قبل commit، يجب أن يتم rollback لكل الخطوات السابقة.

Atomic Stock Update

لحماية المخزون، لن نعتمد على أسلوب read ثم write، كما كنا نستخدمه مع القفل قبل الآن لأنه يسبب بطئ في الأداء ويقال من التوازي بجعل الطلبات تنتظر لحين انتهاء المعاملة المركبة التي تأخذ وقتاً طويلاً، بدلاً من ذلك سنستخدم atomic update : (كود SQL)

```
UPDATE products
SET stock = stock - :quantity
WHERE id = :productId
AND active = true
AND stock >= :quantity;
```

إذا كان عدد الصفوف المتأثرة = 1، فهذا يعني أن المخزون كان كافياً وتم إنقاؤه بنجاح.
 إذا كان عدد الصفوف المتأثرة = 0، فهذا يعني أن المنتج غير موجود أو غير فعال أو أن المخزون غير كافٍ، وبالتالي يتم رفض الطلب.
 هذا التصميم أفضل من read-then-write لأن التحقق من stock وتخفيضه يحدثان في عملية واحدة داخل قاعدة البيانات.

مستويات العزل Isolation Levels

تمت دراسة مستويات العزل التالية، التي تتم داخل المعاملات:

- READ UNCOMMITTED :

يسمح بقراءة بيانات لم يتم عمل commit لها بعد. هذا قد يؤدي إلى dirty read. هذا المستوى غير مناسب للعمليات التجارية مثل checkout لأنه قد يجعل transaction تقرأ بيانات قد يتم التراجع عنها لاحقاً.

- READ COMMITTED :

يمنع dirty reads، لأن transaction لا ترى إلا بيانات committed. لكنه قد يسمح بقراءات غير متكررة أو phantom reads. هذا المستوى مناسب لكثير من العمليات العادية، ويمكن

استخدامه في **checkout** إذا كان تحديث **stock** محمياً بـ **atomic update**، وبالتالي هذا النوع مناسب لحالة الاستخدام لدينا.

- REPEATABLE READ

يعطي **snapshot** أكثر ثباتاً داخل **transaction** في **MySQL InnoDB** هو المستوى الافتراضي.

مناسب عندما نحتاج أن تكون القراءات داخل **transaction** أكثر ثباتاً، لكنه لا يعني تلقائياً أن مشكلة **stock** انتهت إذا استخدمنا **read-then-write** بطريقة خاطئة.

- SERIALIZABLE

الأقوى منطقياً، حيث يجعل النتيجة كأن المعاملات نُفذت واحدة بعد الأخرى. لكنه قد يؤدي إلى تقليل التوازي وزيادة **lock waiting** وارتفاع **p95** و **p99**. **لذلك لا نستخدمه** كحل عام لكل النظام، بل فقط لحالات **حرجة جداً** إذا أثبتت التجارب الحاجة إليه.

القرار النهائي:

لا نعتمد على **isolation** وحده لحماية المخزون. نستخدم **transaction** واضحة **atomic stock** + **update** + **constraints**.

لماذا لا نضع الدفع الخارجي داخل Transaction ؟

في هذا المشروع لا نطبق **Payment Provider** حقيقي، لكننا نحاكي وجود نظام خارجي عبر **payment_attempt**. السبب أن الأنظمة الخارجية لا تشارك في **rollback** الخاص بقاعدة البيانات. لو وضعنا استدعاء خدمة خارجية داخل **transaction** ، قد يحدث التالي:

- يتم استدعاء الخدمة الخارجية بنجاح.
- ثم تفشل قاعدة البيانات قبل **commit**.
- النتيجة: النظام الخارجي نفذ العملية، لكن قاعدة البيانات لا تحتوي على حالة صحيحة.

أو العكس:

- قاعدة البيانات نجحت.
- لكن الاتصال الخارجي فشل.
- النتيجة **order**: موجود، لكن المعالجة الخارجية لم تتم.

لذلك في مشروعنا سنبقى **transaction** الرئيسية محصورة في قاعدة البيانات، ونمثل النظام الخارجي عبر **payment_attempt** و **outbox_event**.

Outbox Event داخل Transaction

من الخطأ إرسال رسالة إلى **RabbitMQ** أو **email** مباشرة داخل **transaction**. لذلك نستخدم **outbox_event**.

داخل **transaction**:

- يتم إنشاء order.
- يتم إنشاء outbox_event بحالة PENDING.
- بعد **commit**، يمكن لمكوّن آخر لاحقاً قراءة outbox_events ونشر الرسائل أو تنفيذ محاكاة خارجية.

الميزة هنا:

- إذا فشلت **transaction**، لا يوجد order ولا outbox_event.
- إذا نجحت **transaction**، يوجد order ويوجد event قابل للمعالجة.
- هذا نضمن أن الأحداث الخارجية لا تسبق نجاح قاعدة البيانات.

Failure Scenarios لإثبات Transaction Integrity:

لإثبات أن **transaction** تعمل بشكل صحيح، سنقوم بإنشاء حالات فشل مقصودة.

1 Failure: فشل بعد تحديث stock

السيناريو:

- Stock قبل الطلب = 10
- يتم تنفيذ atomic update فينقص stock
- ثم نرمي RuntimeException قبل إنشاء order المتوقع مع transaction:
- Stock يرجع إلى 10
- لا يتم إنشاء order
- لا يتم إنشاء order_item
- لا يتم إنشاء payment_attempt
- لا يتم إنشاء outbox_event

بدون: transaction

- Stock قد يصبح 9 رغم فشل الطلب
- هذا يثبت أهمية **Atomicity**.

2 Failure: فشل بعد إنشاء order

السيناريو:

- يتم إنقاص stock.
- يتم إنشاء order.
- ثم يحدث Exception قبل إنشاء payment_attempt.
- المتوقع:

- rollback لكل شيء.
- لا stock ناقص.
- لا order محفوظ.

• لا payment_attempt

3 Failure: فشل بعد إنشاء payment_attempt

السيناريو:

- stock update نجح.
 - Order حفظ.
 - payment_attempt حفظ.
 - ثم Exception.
- المتوقع:
- Rollback لكل التغييرات.
 - لا تبقى payment_attempt منفصلة عن order.

4 Failure: فشل بعد outbox_event

السيناريو:

- كل شيء تم حفظه.
 - بعد إنشاء outbox_event يحدث Exception قبل نهاية method.
- المتوقع:
- لا order.
 - لا order_item.
 - لا payment_attempt.
 - لا outbox_event.

5 Failure: ضغط متزامن على منتج محدود المخزون

السيناريو:

- initial stock = 50
 - عدد المستخدمين المتزامنين = 100
 - كل مستخدم يطلب quantity = 1
- المتوقع:
- successful orders = 50
 - rejected orders = 50
 - final stock = 0
 - لا يوجد stock سالب
 - لا يوجد order بدون order_items

وهنا نتحقق من قاعدة مهمة:

initialStock - finalStock = sum(successful order quantities)

أي إذا تحققت هذه المعادلة، فهذا يدل على أن checkout يحافظ على consistency تحت الضغط.

الاختبارات قبل إضافة ال-Transaction وبعد اضافته:

GET <http://localhost:8080/integrity/product/1> Send

Params Authorization Headers (7) Body Scripts Settings Cookies

Query Params

Key	Value	Description
Key	Value	Description

200 OK • 696 ms • 256 B Save Response

JSON Preview Visualize

```
1 {
2   "currentStock": 1977,
3   "orphanOrderItems": 0,
4   "soldQuantity": 0,
5   "orphanPayments": 0,
6   "productId": 1
7 }
```

هنا تم التأكد من قيم العمليات المركبة التي تمت على في تابع

GET <http://localhost:8080/integrity/product/1> Send

Params Authorization Headers (7) Body Scripts Settings Cookies

Query Params

Key	Value	Description
Key	Value	Description

200 OK • 37 ms • 256 B Save Response

JSON Preview Visualize

```
1 {
2   "orphanPayments": 0,
3   "productId": 1,
4   "currentStock": 1976,
5   "orphanOrderItems": 0,
6   "soldQuantity": 0
7 }
```

Ecommerce Spring Boot / Transaction / Rollback

POST <http://localhost:8080/orders/checkout/fail-after-stock-with-transaction> 

Send

Docs Params Authorization Headers (10) Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {"userId":1,"productId":1,"quantity":1,"paymentMethod":"SIMULATED"}
```

Body Cookies Headers (4) Test Results  500 Internal Server Error 230 ms 304 B   Save Response 

{ } JSON Preview  Debug with AI 

```
1 {
2   "timestamp": "2026-06-21T07:55:32.921+00:00",
3   "status": 500,
4   "error": "Internal Server Error",
5   "path": "/orders/checkout/fail-after-stock-with-transaction"
6 }
```



Ecommerce Spring Boot / Transaction / Integrity Check

GET http://localhost:8080/integrity/product/1

Send

Docs Params Authorization Headers (7) Body Scripts Settings

Query Params

Key	Value	Description
Key	Value	Description

Body Cookies Headers (5) Test Results

200 OK · 68 ms · 256 B

JSON Preview Visualize

```
1 {
2   "orphanPayments": 0,
3   "productId": 1,
4   "currentStock": 1976,
5   "orphanOrderItems": 0,
6   "soldQuantity": 0
7 }
```

Ecommerce Spring Boot / Transaction / Checkout

POST http://localhost:8080/orders/checkout

Send

Docs Params Authorization Headers (10) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

Schema Beautify

```
1 {"userId":1,"productId":1,"quantity":2,"paymentMethod":"SIMULATED"}
```

Body Cookies Headers (5) Test Results

201 Created · 1.67 s · 236 B

JSON Preview Visualize

```
1 {
2   "orderId": 1006,
3   "userId": 1,
4   "status": "CREATED",
5   "totalAmount": 103.70
6 }
```

Ecommerce Spring Boot / Transaction / Integrity Check

GET http://localhost:8080/integrity/product/1

Send

Docs Params Authorization Headers (7) Body Scripts Settings

Query Params

Key	Value	Description
Key	Value	Description

Body Cookies Headers (5) Test Results

200 OK · 23 ms · 256 B

JSON Preview Visualize

```
1 {
2   "orphanPayments": 1,
3   "productId": 1,
4   "currentStock": 1974,
5   "orphanOrderItems": 1,
6   "soldQuantity": 2
7 }
```

خاتمة ما قمنا به في الطلب الثامن

في هذا الطلب تم التركيز على سلامة المعاملات داخل عملية **checkout**، وهي واحدة من أهم العمليات في نظام التجارة الإلكترونية. الهدف لم يكن فقط إضافة **@Transactional**، بل تحديد **boundary** صحيح للمعاملة، بحيث تشمل كل تغييرات قاعدة البيانات التي يجب أن تنجح أو تفشل معاً.

تم انشاء تابع **placeOrder** كمكان رئيسي للـ **transaction** لأنه يمثل **use case** كامل، وليس مجرد عملية **repository** منفردة. داخل هذه **transaction** يتم إنقاص المخزون باستخدام **atomic update**، ومن ثم إنشاء **order_item** و **order** و **payment_attempt** و **outbox_event**. إذا فشل أي جزء قبل **commit**، يتم **rollback** لكل التغييرات، مما يمنع بقاء قاعدة البيانات في حالة جزئية.

كما تمت دراسة مستويات العزل وسبب عدم الاعتماد على **isolation level** وحده لحماية المخزون. بدلاً من رفع النظام كله إلى **SERIALIZABLE**، تم استخدام **atomic update** لحماية نقطة التنافس الأساسية، مع الحفاظ على **transaction** قصيرة ومحددة.

تم أيضاً تمثيل الأنظمة الخارجية بشكل مبسط عبر **payment_attempt** و **outbox_event**، مع التأكيد على أن الخدمات الخارجية مثل الدفع أو الإشعار لا يجب أن تعمل داخل **transaction** الرئيسية، لأن **rollback** الخاص بقاعدة البيانات لا يمكنه التراجع عن عملية خارجية.

بهذا يكون الطلب الثامن قد ربط بين سلامة البيانات، التزامن، العزل، القيود، والقياس. والنتيجة هي تصميم **checkout** يحافظ على قاعدة البيانات في حالة صحيحة حتى عند حدوث فشل أو ضغط متزامن.

الطلب التاسع : اختبار الاستقرار تحت الضغط

الهدف التحقق من قدرة النظام على التعامل مع عدد كبير من المستخدمين المتزامنين دون حدوث انهيار أو فقدان للبيانات والتأكد من استقرار الأداء أثناء تنفيذ العمليات

تم انشاء خطة اختبار في **jmeter** تضمنت أربعة Requests تمثل اهم العمليات في النظام :

- 1- جلب المنتجات
- 2- اضافة منتج
- 3- شراء منتج
- 4- عملية الدفع (checkout)

اعدادات الاختبار :

Threads	Request
30	Get products
20	Add product
30	Buy product
20	Check out

اجمالي المستخدمين المتزامنين = 100 مستخدم

النتائج :

Test-Plan.jmx (C:\Users\enter\OneDrive\Desktop\Test-Plan.jmx) - Apache JMeter (5.6.3)

File Edit Search Run Options Tools Help

00:00:18 0 0/100

Test Plan

- Thread Group
 - Get products
 - View Results Tree
 - Thread Group
 - Add Products
 - Thread Group
 - Buy
 - Thread Group
 - Checkout

View Results in Table

Name: View Results in Table

Comments:

Write results to file / Read from file

Filename: Browse... Log/Display Only: ☐ Errors ☐ Successes Configure

Sample #	Start Time	Thread Name	Label	Sample Time...	Status	Bytes	Sent Bytes	Latency	Connect Time...
1	01:44:08.824	Thread Group ...	Add Products	342	✓	416	243	342	1
2	01:44:08.880	Thread Group ...	Add Products	867	✓	416	243	786	1
3	01:44:08.937	Thread Group ...	Add Products	1047	✓	416	243	853	2
4	01:44:09.232	Thread Group ...	Add Products	1364	✓	416	243	1364	2
5	01:44:08.988	Thread Group ...	Add Products	1613	✓	416	243	1608	2
6	01:44:08.931	Thread Group ...	Buy	1839	✓	676	237	1838	4
7	01:44:08.988	Thread Group ...	Buy	2090	✓	676	237	2090	1
8	01:44:09.184	Thread Group ...	Add Products	1999	✓	416	243	1999	2
9	01:44:09.137	Thread Group ...	Add Products	2245	✓	416	243	2245	2
10	01:44:09.169	Thread Group ...	Add Products	2515	✓	416	243	2515	2
11	01:44:08.940	Thread Group ...	Get products	3510	✓	295706	125	2895	3
12	01:44:08.892	Thread Group ...	Get products	3683	✓	295706	125	2682	2
13	01:44:08.978	Thread Group ...	Buy	3601	✓	676	237	3601	2
14	01:44:08.760	Thread Group ...	Get products	4206	✓	295467	125	2423	1
15	01:44:09.379	Thread Group ...	Add Products	3672	✓	416	243	3672	1
16	01:44:09.578	Thread Group ...	Add Products	3574	✓	416	243	3573	3
17	01:44:09.329	Thread Group ...	Add Products	3945	✓	416	243	3945	2
18	01:44:09.614	Thread Group ...	Add Products	3667	✓	416	243	3660	1
19	01:44:09.037	Thread Group ...	Add Products	4313	✓	416	243	3823	1
20	01:44:09.127	Thread Group ...	Buy	4528	✓	676	237	4528	1

☐ Scroll automatically? ☐ Child samples? No of Samples: 100 Latest Sample: 16792 Average: 3014 Deviation: 4403

Test-Plan.jmx (C:\Users\enter\OneDrive\Desktop\Test-Plan.jmx) - Apache JMeter (5.6.3)

File Edit Search Run Options Tools Help

00:00:18 0 0/100

Test Plan

- Thread Group
 - Get products
 - View Results Tree
- Thread Group
 - Add Products
- Thread Group
 - Buy
- Thread Group
 - Checkout
- View Results in Table

View Results in Table

Name: View Results in Table

Comments:

Write results to file / Read from file

Filename: Browse... Log/Display Only: ☐ Errors ☐ Successes Configure

Sample #	Start Time	Thread Name	Label	Sample Time(...)	Status	Bytes	Sent Bytes	Latency	Connect Time(...)
81	01:44:09.396	Thread Group ...	Buy	13344	✓	676	237	13329	2
82	01:44:09.892	Thread Group ...	Buy	12901	✓	676	237	12901	2
83	01:44:09.932	Thread Group ...	Buy	12934	✓	676	237	12934	2
84	01:44:09.516	Thread Group ...	Buy	13466	✓	676	237	13466	2
85	01:44:09.639	Thread Group ...	Buy	13439	✓	676	237	13439	6
86	01:44:09.310	Thread Group ...	Buy	13977	✓	676	237	13977	3
87	01:44:08.887	Thread Group ...	Checkout	14535	✓	676	261	14541	3
88	01:44:09.255	Thread Group ...	Checkout	14333	✓	676	261	14351	4
89	01:44:09.122	Thread Group ...	Checkout	14835	✓	676	261	14808	1
90	01:44:09.033	Thread Group ...	Checkout	15119	✓	676	261	15105	1
91	01:44:09.189	Thread Group ...	Checkout	15163	✓	675	261	15163	1
92	01:44:09.769	Thread Group ...	Checkout	14714	✓	676	261	14703	2
93	01:44:09.847	Thread Group ...	Checkout	14810	✓	676	261	14804	2
94	01:44:09.703	Thread Group ...	Checkout	15172	✓	675	261	15172	1
95	01:44:09.622	Thread Group ...	Checkout	15726	✓	676	261	15726	2
96	01:44:09.390	Thread Group ...	Checkout	16201	✓	676	261	16185	2
97	01:44:09.659	Thread Group ...	Checkout	16232	✓	676	261	16219	2
98	01:44:09.323	Thread Group ...	Checkout	16876	✓	676	261	16867	3
99	01:44:09.456	Thread Group ...	Checkout	17015	✓	676	261	17015	3
100	01:44:09.948	Thread Group ...	Checkout	16792	✓	676	261	16792	2

☐ Scroll automatically? ☐ Child samples? No of Samples 100 Latest Sample 16792 Average 9314 Deviation 4403

أنشاء تنفيذ الاختبار :

- لم يحدث أي انهيار للخادم .
- جميع الطلبات تمت معالجتها بنجاح .
- لم يتم ملاحظة فقدان للبيانات .
- بقي النظام مستقرا طوال فترة لاختبار .

أثبت اختبار الضغط باستخدام Apache JMeter أن النظام قادر على التعامل مع المستخدمين المتزامنين الذين تم اختبارهم مع الحفاظ على استقرار الخدمة وعدم فقدان البيانات أو توقف النظام، كما حقق جميع الطلبات نسبة نجاح مرتفعة مع أزمنة استجابة مقبولة.

الطلب العاشر : Benchmarking & Bottleneck Analysis

Benchmarking هو عملية قياس مقاييس أداء محددة وقابلة للتكرار تحت ظروف معروفة ومضبوطة بهدف الحصول على أرقام موضوعية تصف سلوك النظام . يختلف عن stress testing الذي عملناه في المتطلب التاسع الذي يبحث عن نقطة الانهيار تحت الضغط المتزايد بينما Benchmarking يقيس الاداء الدقيق لعملية واحدة محددة تحت ظروف ثابتة لايجاد فرص التحسن الفعلية .

Bottleneck Analysis مبني على Theory of Constraints التي طورها Eliyahu Goldratt في أي نظام مكون من خطوات متسلسلة، الأداء الكلي محكوم بأبطأ خطوة فيه. تحسين الخطوات السريعة لا يحسن النظام كله، فقط تحسين الـ bottleneck يفعل ذلك.

الهدف:

قياس أداء عمليات الدفع في النظام وتحديد الاختناقات الرئيسية، من خلال المقارنة بين أسلوبين في معالجة الدفع: المعالجة المتزامنة (Synchronous) والمعالجة غير المتزامنة (Asynchronous with Queue)، وذلك تحت ضغط 100 طلب متزامن باستخدام Apache JMeter

الاسلوب الأول : sync payment

طلب الشراء :

transaction @ -1

findByIdwithLock -2

ProcessPaymentSync -3

← Thread.sleep(3000) جوا Transaction

save Product -4

-5 Transactional خلصت

=> رجع الطلب

الأسلوب الثاني : Async Payment (queue)

طلب الشراء :

transaction @ -1

findByIdwithLock -2

save Product -3

processPaymentAsync → send to RabbitMQ Queue -4

-5 Transactional خلصت

الطلب رجع بسرعة لان عملية الدفع تمت بالخلفية عن طريق الـ consumer وهي الـ thread.sleep(3000)

الاختناق Bottleneck

كان يحدث في تابع الـ processPaymentSync عندما كان يتم معالجة الدفع فيه والذي يستغرق 3 ثواني والمستخدم ينتظر حتى تنتهي كل العمليات ليرجع له الطلب .

نتائج الطلبين

Sync -1

Sample #	Start Time	Thread Name	Label	Sample Time(...)	Status	Bytes	Sent Bytes	Latency	Connect Time(...)
1	13:13:23.552	Thread Group 1...	Checkout	27802	✓	695	260	27659	2
2	13:13:24.319	Thread Group 1...	Checkout	29389	✓	695	260	29389	2
3	13:13:23.814	Thread Group 1...	Checkout	33038	✓	695	260	33037	1
4	13:13:23.710	Thread Group 1...	Checkout	36390	✓	695	260	36390	1
5	13:13:23.507	Thread Group 1...	Checkout	40163	✓	695	260	40119	2
6	13:13:24.001	Thread Group 1...	Checkout	42868	✓	695	260	42868	1
7	13:13:24.199	Thread Group 1...	Checkout	45873	✓	695	260	45873	2
8	13:13:23.879	Thread Group 1...	Checkout	49616	✓	695	260	49583	4
9	13:13:23.424	Thread Group 1...	Checkout	53313	✓	695	260	53313	21
10	13:13:24.115	Thread Group 1...	Checkout	55718	✓	695	260	55717	2

```

- Started: ProductService.buyWithPaymentSync
- Started: PaymentService.processPaymentSync
[PAYMENT-SYNC] Processing payment for: Customer 2 | Amount: 8.47
[PAYMENT-SYNC] Payment approved for: Customer 2 | New Balance: 999822.1300000006
- Finished: PaymentService.processPaymentSync | Time: 3327ms
- Finished: ProductService.buyWithPaymentSync | Time: 5104ms

```

هنا ال buyWithPaymentSync لم تنتهي الى ان انتهت ال processPaymentSync

Async -2

Sample #	Start Time	Thread Name	Label	Sample Time(...)	Status	Bytes	Sent Bytes	Latency	Connect Time(...)
1	12:36:37.001	Thread Group 1...	Checkout	19052	✓	695	261	18793	1
2	12:36:36.796	Thread Group 1...	Checkout	19272	✓	695	261	18983	2
3	12:36:37.401	Thread Group 1...	Checkout	18667	✓	695	261	18298	2
4	12:36:37.303	Thread Group 1...	Checkout	18765	✓	695	261	18491	2
5	12:36:37.508	Thread Group 1...	Checkout	19132	✓	694	261	19132	2
6	12:36:36.898	Thread Group 1...	Checkout	21799	✓	695	261	21799	2
7	12:36:37.112	Thread Group 1...	Checkout	25013	✓	695	261	25010	2
8	12:36:36.620	Thread Group 1...	Checkout	28811	✓	695	261	28811	18
9	12:36:37.201	Thread Group 1...	Checkout	31022	✓	695	261	31007	1
10	12:36:36.698	Thread Group 1...	Checkout	33328	✓	695	261	33328	3

وتم ايضا مراقبة كلا الطلبين باستخدام Grafana في ال Docker